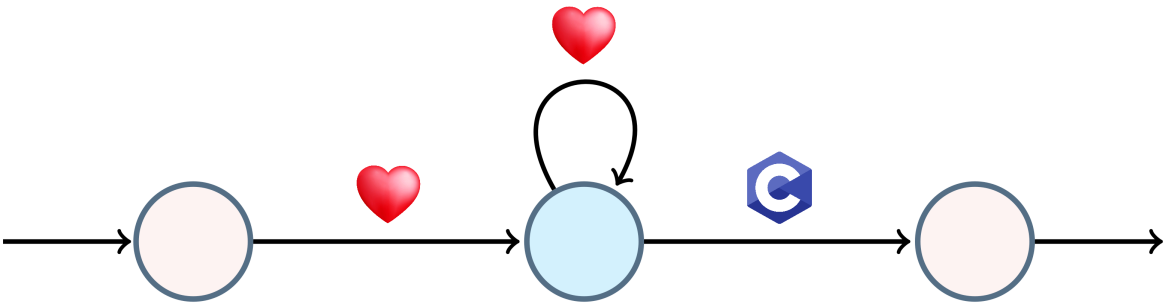
Projet C : Automates



Généré par Doxygen 1.15.0

1 Index des structures de données **1**

1.1 Structures de données 1

2 Index des fichiers **3**

2.1 Liste des fichiers 3

3 Documentation des structures de données **5**

3.1 Référence de la structure barray 5

3.1.1 Description détaillée 5

3.1.2 Documentation des champs 6

3.1.2.1 array 6

3.1.2.2 size 6

3.1.2.3 size_array 6

3.2 Référence de la structure hash_table 6

3.2.1 Description détaillée 7

3.2.2 Documentation des champs 7

3.2.2.1 hash_capacity 7

3.2.2.2 hash_size 8

3.2.2.3 hash_table 8

3.2.2.4 element_size 8

3.3 Référence de la structure multi_edge 8

3.3.1 Description détaillée 9

3.3.2 Documentation des champs 9

3.3.2.1 in 9

3.3.2.2 lab 9

3.3.2.3 out 9

3.4 Référence de la structure nfa 10

3.4.1 Description détaillée 11

3.4.2 Documentation des champs 11

3.4.2.1 nb_states 11

3.4.2.2 nb_letters 11

3.4.2.3 eps_trans 11

3.4.2.4 transitions 12

3.4.2.5 initials 12

3.4.2.6 nb_initials 12

3.4.2.7 finals 12

3.4.2.8 nb_finals 12

3.4.2.9 letter_names 13

3.4.2.10 state_names . . . 13
3.5 Référence de la structure ngraph . . . 13
3.5.1 Description détaillée . . . 14
3.5.2 Documentation des champs . . . 14
3.5.2.1 nb_states . . . 14
3.5.2.2 nb_labels . . . 14
3.5.2.3 to_array . . . 14
3.5.2.4 intervals . . . 15
3.6 Référence de la structure one_edge . . . 15
3.6.1 Description détaillée . . . 15
3.6.2 Documentation des champs . . . 15
3.6.2.1 from . . . 15
3.6.2.2 to . . . 16
3.6.2.3 label . . . 16
3.7 Référence de la structure regexp . . . 16
3.7.1 Description détaillée . . . 17
3.7.2 Documentation des champs . . . 17
3.7.2.1 op . . . 17
3.7.2.2 left . . . 17
3.7.2.3 right . . . 17
3.7.2.4 letter . . . 17

4 Documentation des fichiers **19**

4.1 Référence du fichier alloc.c . . . 19
4.1.1 Description détaillée . . . 19
4.1.2 Documentation des fonctions . . . 20
4.1.2.1 multiple_free() . . . 20
4.1.2.2 check_null() . . . 20
4.2 alloc.c . . . 21
4.3 Référence du fichier alloc.h . . . 21
4.3.1 Description détaillée . . . 22
4.3.2 Documentation des macros . . . 23
4.3.2.1 CREATE_MEMORY . . . 23
4.3.2.2 CREATE_ZERO_MEMORY . . . 24
4.3.2.3 REDIMENSION_MEMORY . . . 25
4.3.2.4 CHECK_NULL . . . 26
4.3.3 Documentation des fonctions . . . 26
4.3.3.1 check_null() . . . 26

4.3.3.2 multiple_free() . . . 27
4.4 alloc.h . . . 27
4.5 Référence du fichier error.h . . . 28
4.5.1 Description détaillée . . . 30
4.5.2 Documentation des macros . . . 31
4.5.2.1 ATTENTION . . . 31
4.5.2.2 HINT . . . 31
4.5.2.3 OK . . . 31
4.5.2.4 KO . . . 31
4.5.2.5 DEBUG_LEVEL . . . 32
4.5.2.6 PRINT_ERROR . . . 32
4.5.2.7 FATAL . . . 34
4.5.2.8 PRINT_DEBUG . . . 34
4.5.2.9 TRACE . . . 35
4.5.2.10 DEBUG . . . 35
4.5.2.11 INFO . . . 35
4.5.2.12 WARNING . . . 36
4.5.2.13 ERROR . . . 36
4.5.2.14 CRITICAL . . . 36
4.5.3 Documentation du type de l'énumération . . . 36
4.5.3.1 debug_level . . . 36
4.5.3.2 msg_level . . . 37
4.6 error.h . . . 37
4.7 Référence du fichier graphs.c . . . 39
4.7.1 Documentation des fonctions . . . 39
4.7.1.1 ngraph_to_edge_list() . . . 39
4.7.1.2 edge_list_to_ngraph() . . . 40
4.7.1.3 ngraph_to_multi_edges() . . . 41
4.8 graphs.c . . . 41
4.9 Référence du fichier graphs.h . . . 42
4.9.1 Description détaillée . . . 43
4.9.2 Documentation des définitions de type . . . 43
4.9.2.1 ngraph . . . 43
4.9.3 Documentation des fonctions . . . 44
4.9.3.1 ngraph_to_edge_list() . . . 44
4.9.3.2 edge_list_to_ngraph() . . . 44
4.9.3.3 ngraph_to_multi_edges() . . . 45

4.10 graphs.h . . . 45
4.11 Référence du fichier grep.c . . . 47
4.11.1 Documentation des fonctions . . . 47
4.11.1.1 grep() . . . 47
4.12 grep.c . . . 47
4.13 Référence du fichier grep.h . . . 48
4.13.1 Description détaillée . . . 48
4.13.2 Documentation des fonctions . . . 48
4.13.2.1 grep() . . . 48
4.14 grep.h . . . 49
4.15 Référence du fichier nfa.c . . . 49
4.15.1 Documentation des fonctions . . . 50
4.15.1.1 nfa_reset_state_names() . . . 50
4.15.1.2 create_emptylang() . . . 50
4.15.1.3 create_sing_epsilon() . . . 51
4.15.1.4 create_sing_letter() . . . 51
4.15.1.5 nfa_delete() . . . 52
4.15.1.6 nfa_copy() . . . 52
4.15.1.7 nfa_union() . . . 52
4.15.1.8 nfa_concat() . . . 53
4.15.1.9 nfa_star() . . . 53
4.15.1.10 nfa_trim() . . . 53
4.15.1.11 nfa_mirror() . . . 54
4.15.1.12 nfa_elimeps() . . . 54
4.15.1.13 nfa_compute_runs() . . . 54
4.16 nfa.c . . . 55
4.17 Référence du fichier nfa.h . . . 56
4.17.1 Description détaillée . . . 58
4.17.2 Documentation des fonctions . . . 58
4.17.2.1 nfa_reset_state_names() . . . 58
4.17.2.2 create_emptylang() . . . 58
4.17.2.3 create_sing_epsilon() . . . 59
4.17.2.4 create_sing_letter() . . . 59
4.17.2.5 nfa_delete() . . . 59
4.17.2.6 nfa_copy() . . . 60
4.17.2.7 nfa_union() . . . 60
4.17.2.8 nfa_concat() . . . 60

4.17.2.9 nfa_star() . . . 61
4.17.2.10 nfa_trim() . . . 61
4.17.2.11 nfa_mirror() . . . 61
4.17.2.12 nfa_elimeps() . . . 62
4.17.2.13 nfa_compute_runs() . . . 62
4.18 nfa.h . . . 63
4.19 Référence du fichier nfa_determi.c . . . 64
4.19.1 Documentation des fonctions . . . 65
4.19.1.1 nfa_determinize() . . . 65
4.19.1.2 nfa_complement() . . . 65
4.20 nfa_determi.c . . . 66
4.21 Référence du fichier nfa_determi.h . . . 66
4.21.1 Description détaillée . . . 67
4.21.2 Documentation des fonctions . . . 67
4.21.2.1 nfa_determinize() . . . 67
4.21.2.2 nfa_complement() . . . 68
4.22 nfa_determi.h . . . 68
4.23 Référence du fichier nfa_intersec.c . . . 68
4.23.1 Documentation des fonctions . . . 69
4.23.1.1 nfa_intersect() . . . 69
4.24 nfa_intersec.c . . . 70
4.25 Référence du fichier nfa_intersec.h . . . 70
4.25.1 Description détaillée . . . 70
4.25.2 Documentation des fonctions . . . 71
4.25.2.1 nfa_intersect() . . . 71
4.26 nfa_intersec.h . . . 72
4.27 Référence du fichier nfa_mccluskey.c . . . 72
4.27.1 Documentation des fonctions . . . 72
4.27.1.1 nfa_mccluskey() . . . 72
4.28 nfa_mccluskey.c . . . 73
4.29 Référence du fichier nfa_mccluskey.h . . . 73
4.29.1 Description détaillée . . . 73
4.29.2 Documentation des fonctions . . . 73
4.29.2.1 nfa_mccluskey() . . . 73
4.30 nfa_mccluskey.h . . . 74
4.31 Référence du fichier nfa_minimization.c . . . 74
4.31.1 Documentation des fonctions . . . 74

4.31.1.1 nfa_brzozowski() . . . 74
4.32 nfa_minimization.c . . . 75
4.33 Référence du fichier nfa_minimization.h . . . 75
4.33.1 Description détaillée . . . 75
4.33.2 Documentation des fonctions . . . 76
4.33.2.1 nfa_brzozowski() . . . 76
4.34 nfa_minimization.h . . . 76
4.35 Référence du fichier parameters.h . . . 76
4.35.1 Documentation des macros . . . 77
4.35.1.1 ADVANCED . . . 77
4.36 parameters.h . . . 77
4.37 Référence du fichier regexp.c . . . 77
4.37.1 Documentation des fonctions . . . 78
4.37.1.1 reg_issimple() . . . 78
4.37.1.2 reg_free() . . . 79
4.37.1.3 reg_copy() . . . 79
4.37.1.4 reg_empty() . . . 79
4.37.1.5 reg_epsilon() . . . 80
4.37.1.6 reg_letter() . . . 80
4.37.1.7 reg_union() . . . 80
4.37.1.8 reg_inter() . . . 81
4.37.1.9 reg_concat() . . . 81
4.37.1.10 reg_star() . . . 81
4.37.1.11 reg_complement() . . . 82
4.37.1.12 reg_print_helper() . . . 82
4.37.1.13 reg_print() . . . 82
4.38 regexp.c . . . 83
4.39 Référence du fichier regexp.h . . . 86
4.39.1 Description détaillée . . . 88
4.39.2 Documentation des définitions de type . . . 88
4.39.2.1 regexp . . . 88
4.39.3 Documentation du type de l'énumération . . . 88
4.39.3.1 regelem . . . 88
4.39.4 Documentation des fonctions . . . 89
4.39.4.1 reg_issimple() . . . 89
4.39.4.2 reg_free() . . . 89
4.39.4.3 reg_copy() . . . 89

4.39.4.4 reg_empty() . . . 90
4.39.4.5 reg_epsilon() . . . 90
4.39.4.6 reg_letter() . . . 90
4.39.4.7 reg_union() . . . 90
4.39.4.8 reg_inter() . . . 91
4.39.4.9 reg_concat() . . . 91
4.39.4.10 reg_star() . . . 91
4.39.4.11 reg_complement() . . . 92
4.39.4.12 reg_print() . . . 92
4.40 regexp.h . . . 92
4.41 Référence du fichier regexp_thompson.c . . . 94
4.41.1 Documentation des fonctions . . . 94
4.41.1.1 reg_thompson() . . . 94
4.42 regexp_thompson.c . . . 95
4.43 Référence du fichier regexp_thompson.h . . . 95
4.43.1 Description détaillée . . . 95
4.43.2 Documentation des fonctions . . . 95
4.43.2.1 reg_thompson() . . . 95
4.44 regexp_thompson.h . . . 96
4.45 Référence du fichier type_basic.h . . . 96
4.45.1 Description détaillée . . . 97
4.45.2 Documentation des macros . . . 97
4.45.2.1 max . . . 97
4.45.2.2 min . . . 97
4.45.3 Documentation des définitions de type . . . 98
4.45.3.1 uchar . . . 98
4.45.3.2 ushort . . . 98
4.45.3.3 uint . . . 98
4.45.3.4 ulong . . . 98
4.46 type_basic.h . . . 98
4.47 Référence du fichier type_boolarray.c . . . 99
4.47.1 Documentation des fonctions . . . 100
4.47.1.1 create_barray() . . . 100
4.47.1.2 getsize_barray() . . . 100
4.47.1.3 delete_barray() . . . 101
4.47.1.4 settrue_barray() . . . 101
4.47.1.5 setfalse_barray() . . . 101

4.47.1.6 getval_barray() . . . 101
4.47.1.7 or_barray() . . . 102
4.47.1.8 and_barray() . . . 102
4.47.1.9 copy_barray() . . . 102
4.48 type_boolarray.c . . . 103
4.49 Référence du fichier type_boolarray.h . . . 103
4.49.1 Description détaillée . . . 104
4.49.2 Documentation des fonctions . . . 105
4.49.2.1 create_barray() . . . 105
4.49.2.2 getsize_barray() . . . 105
4.49.2.3 delete_barray() . . . 105
4.49.2.4 settrue_barray() . . . 106
4.49.2.5 setfalse_barray() . . . 106
4.49.2.6 getval_barray() . . . 106
4.49.2.7 or_barray() . . . 106
4.49.2.8 and_barray() . . . 107
4.49.2.9 copy_barray() . . . 107
4.50 type_boolarray.h . . . 107
4.51 Référence du fichier type_hash.c . . . 108
4.51.1 Documentation des fonctions . . . 109
4.51.1.1 create_hash_table() . . . 109
4.51.1.2 delete_hash_table() . . . 109
4.51.1.3 hash_function() . . . 110
4.51.1.4 hash_table_rehash() . . . 111
4.51.1.5 hash_array_compare() . . . 111
4.51.1.6 hash_table_insert() . . . 111
4.52 type_hash.c . . . 112
4.53 Référence du fichier type_hash.h . . . 114
4.53.1 Description détaillée . . . 115
4.53.2 Documentation des fonctions . . . 115
4.53.2.1 create_hash_table() . . . 115
4.53.2.2 delete_hash_table() . . . 116
4.53.2.3 hash_table_insert() . . . 116
4.54 type_hash.h . . . 117

Index **119**

Chapitre 1

Index des structures de données

1.1 Structures de données

Liste des structures de données avec une brève description :

barray
: Type utilisé pour représenter un tableau de Booléens 5

hash_table
: Type utilisé pour représenter une table de hachage 6

multi_edge
: Type utilisé pour représenter une multi-arête 8

nfa
: Type utilisé pour représenter un NFA 10

ngraph
: Type utilisé pour représenter un graphe orienté et étiqueté 13

one_edge
: Type utilisé pour représenter une unique arrête d'un graphe étiqueté 15

regexp
: Type utilisé pour représenter un noeud d'une expression régulière 16

Chapitre 2

Index des fichiers

2.1 Liste des fichiers

Liste de tous les fichiers avec une brève description :

alloc.c
: Fonctions d'allocation mémoire . . . 19
alloc.h
: Macros et fonctions d'allocation mémoire . . . 21
error.h
: Macros permettant d'afficher des messages d'erreur . . . 28
graphs.c . . . 39
graphs.h
: Implémentation des graphes étiquetés . . . 42
grep.c . . . 47
grep.h
: Implémentation de la fonction grep . . . 48
nfa.c . . . 49
nfa.h
: Implémentation des automates . . . 56
nfa_determi.c . . . 64
nfa_determi.h
: Déterminisation et complémentation . . . 66
nfa_intersec.c . . . 68
nfa_intersec.h
: Intersection de NFAs . . . 70
nfa_mccluskey.c . . . 72
nfa_mccluskey.h
: Calcul d'une expression régulière à partir d'un NFA avec l'algorithme de Brzozowski-McCluskey . . . 73
nfa_minimization.c . . . 74
nfa_minimization.h
: Minimisation . . . 75
parameters.h . . . 76
regexp.c . . . 77

regexp.h
Implémentation des expressions régulières 86
regexp_thompson.c 94
regexp_thompson.h
Construction d'un NFA à partir d'une expression régulière avec l'algorithme de Thompson 95
type_basic.h
Définitions des types basiques 96
type_boolarray.c 99
type_boolarray.h
Implémentation des tableaux de Booléens 103
type_hash.c 108
type_hash.h
Implémentation des tables de hachage (déjà écrite, à utiliser pour les parcours de graphes) 114

Chapitre 3

Documentation des structures de données

3.1 Référence de la structure barray

Type utilisé pour représenter un tableau de Booléens.

`#include <type_boolarray.h>`

Champs de données

— uchar * array

Le tableau contenant les Booléens.

— int size

Le nombre de Booléens stockés.

— int size_array

La taille réelle du tableau (`size/8` ou `size/8 + 1`, selon la valeur de `size % 8`).

3.1.1 Description détaillée

Type utilisé pour représenter un tableau de Booléens.

Un tableau de Booléens est codé par un tableau de `unsigned char`. Chaque `usigned char` prend un octet en mémoire (8 bits) et peut donc stocker 8 Booléens.

L'implémentation est facilitée par les opérateurs de manipulation **bit à bit** |, &, ~, >> et <<.

Définition à la ligne 23 du fichier type_boolarray.h.

3.1.2 Documentation des champs

3.1.2.1 array

`uchar* array`

Le tableau contenant les Booléens.

Définition à la ligne 25 du fichier type_boolarray.h.

3.1.2.2 size

`int size`

Le nombre de Booléens stockés.

Définition à la ligne 26 du fichier type_boolarray.h.

3.1.2.3 size_array

`int size_array`

La taille réelle du tableau (`size/8` ou `size/8 + 1`, selon la valeur de `size % 8`).

Définition à la ligne 27 du fichier type_boolarray.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— type_boolarray.h

3.2 Référence de la structure hash_table

Type utilisé pour représenter une table de hachage.

`#include <type_hash.h>`

Champs de données

— int hash_capacity

Capacité actuelle de la table de hachage.

— int hash_size

Taille du hachage (nombre d'éléments dans la table de hachage).

— int * hash_table

Pointeur vers la table de hachage elle-même (tableau d'indices vers le tableau de stockage externe).

— int element_size

Nombre de valeurs utilisées pour encoder un seul élément dans le tableau de stockage externe.

3.2.1 Description détaillée

Type utilisé pour représenter une table de hachage.

Utilisée pour stocker les configurations de plateau visitées dans le résolveur. La table de hachage ne stocke pas les objets eux-mêmes, mais des indices vers un tableau de stockage externe qui est passé aux fonctions de la table de hachage (ceci est nécessaire car le tableau de stockage externe peut avoir besoin d'être réalloué et déplacé en mémoire).

Remarques

Chaque élément inséré doit être encodé dans le tableau de stockage externe comme une séquence d'entiers (la longueur de la séquence est fixée à la création de la table de hachage). En particulier, cela signifie que les objets à insérer doivent être encodés comme des tableaux d'entiers.

Définition à la ligne 39 du fichier type_hash.h.

3.2.2 Documentation des champs

3.2.2.1 hash_capacity

```
int hash_capacity
```

Capacité actuelle de la table de hachage.

Définition à la ligne 41 du fichier type_hash.h.

3.2.2.2 hash_size

```
int hash_size
```

Taille du hachage (nombre d'éléments dans la table de hachage).

Définition à la ligne 42 du fichier type_hash.h.

3.2.2.3 hash_table

```
int* hash_table
```

Pointeur vers la table de hachage elle-même (tableau d'indices vers le tableau de stockage externe).

Définition à la ligne 43 du fichier type_hash.h.

3.2.2.4 element_size

```
int element_size
```

Nombre de valeurs utilisées pour encoder un seul élément dans le tableau de stockage externe.

Définition à la ligne 44 du fichier type_hash.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— type_hash.h

3.3 Référence de la structure multi_edge

Type utilisé pour représenter une multi-arête.

```
#include <graphs.h>
```

Champs de données

— int in

État de départ de l'arête.

— barray * lab

Liste des étiquettes.

— int out

État de sortie de l'arête.

3.3.1 Description détaillée

Type utilisé pour représenter une multi-arête.

Une multi-arête relie deux sommets et peut porter plusieurs étiquettes. Utilisé uniquement dans l'affichage avancé des automates pour simplifier la représentation graphique.

Remarques

> Une multi-arête est une arête qui peut avoir plusieurs étiquettes. On se sert de ce type pour l'affichage : quand deux sommets sont reliés par plusieurs arêtes, on simplifie la présentation en ne dessinant qu'une seule arête qui porte toutes les étiquettes des vraies arêtes.

Définition à la ligne 133 du fichier graphs.h.

3.3.2 Documentation des champs

3.3.2.1 in

```
int in
```

État de départ de l'arête.

Définition à la ligne 134 du fichier graphs.h.

3.3.2.2 lab

```
barray* lab
```

Liste des étiquettes.

Définition à la ligne 135 du fichier graphs.h.

3.3.2.3 out

```
int out
```

État de sortie de l'arête.

Définition à la ligne 136 du fichier graphs.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— graphs.h

3.4 Référence de la structure nfa

Type utilisé pour représenter un NFA.

`#include <nfa.h>`

Champs de données

— int nb_states

Le nombre d'états (identique à transitions->nb_states).

— int nb_letters

Le nombre de lettres.

— bool eps_trans

Indique si l'automate possède des transitions epsilon.

— ngraph * transitions

Le graphe des transitions.

— int * initials

Le tableau des états initiaux.

— int nb_initials

Le nombre d'états initiaux.

— int * finals

Le tableau des états finaux.

— int nb_finals

Le nombre d'états finaux.

— char * letter_names

Le tableau des noms des lettres.

— char ** state_names

Le tableau des noms des états (NULL si pas utilisé).

3.4.1 Description détaillée

Type utilisé pour représenter un NFA.

Remarques

Si l'automate ne possède pas de transitions epsilon, alors le champ nb_letters est égal à transitions->nb_labels (les étiquettes du graphe des transitions correspondent exactement aux lettres de l'alphabet de l'automate). Sinon, si il y a des transitions epsilon, alors transitions->nb↩ _labels = nb_letters + 1

- Les étiquettes de 0 à nb_letters-1 correspondent aux lettres de l'alphabet.
- L'étiquette nb_letters correspond aux transitions epsilon.

Le tableau des noms des états n'est pas toujours utilisé (dans ce cas le nom d'un état est simplement son numéro). Si on ne l'utilise pas, il doit avoir la valeur `NULL`. Quand on l'utilise, chaque nom d'état est une chaîne de caractères allouée dynamiquement (il faudra donc penser à libérer ces chaînes ainsi que le tableau des noms des états quand on n'en a plus besoin).

Définition à la ligne 41 du fichier nfa.h.

3.4.2 Documentation des champs

3.4.2.1 nb_states

```
int nb_states
```

Le nombre d'états (identique à transitions->nb_states).

Définition à la ligne 43 du fichier nfa.h.

3.4.2.2 nb_letters

```
int nb_letters
```

Le nombre de lettres.

Définition à la ligne 44 du fichier nfa.h.

3.4.2.3 eps_trans

```
bool eps_trans
```

Indique si l'automate possède des transitions epsilon.

Définition à la ligne 45 du fichier nfa.h.

3.4.2.4 transitions

```
ngraph* transitions
```

Le graphe des transitions.

Définition à la ligne 46 du fichier nfa.h.

3.4.2.5 initials

```
int* initials
```

Le tableau des états initiaux.

Définition à la ligne 47 du fichier nfa.h.

3.4.2.6 nb_initials

```
int nb_initials
```

Le nombre d'états initiaux.

Définition à la ligne 48 du fichier nfa.h.

3.4.2.7 finals

```
int* finals
```

Le tableau des états finaux.

Définition à la ligne 49 du fichier nfa.h.

3.4.2.8 nb_finals

```
int nb_finals
```

Le nombre d'états finaux.

Définition à la ligne 50 du fichier nfa.h.

3.4.2.9 letter_names

`char* letter_names`

Le tableau des noms des lettres.

Définition à la ligne 51 du fichier nfa.h.

3.4.2.10 state_names

`char** state_names`

Le tableau des noms des états (NULL si pas utilisé).

Définition à la ligne 54 du fichier nfa.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— nfa.h

3.5 Référence de la structure ngraph

Type utilisé pour représenter un graphe orienté et étiqueté.

`#include <graphs.h>`

Champs de données

— int nb_states

Nombre de sommets.

— int nb_labels

Taille de l'alphabet étiquetant les transitions.

— int * to_array

— int * intervals

3.5.1 Description détaillée

Type utilisé pour représenter un graphe orienté et étiqueté.

Le graphe contient nb_states sommets, numérotés de 0 à nb_states-1 et il y a nb_labels étiquettes numérotées de 0 à nb_labels-1. Les arrêtes sont codées par les deux tableaux "to_array" et "intervals":

- "intervals" est un tableau de taille "nb_states $*$ nb_labels + 1". Il stocke des indices dans le tableau "to_array" triés par ordre croissant.
- "to_array" est un tableau d'entiers, qui stocke les états d'arrivée des arrêtes (sa taille est égale au nombre total d'arrêtes dans le graphe). Pour chaque état q et chaque étiquette a, les états d'arrivée des arrêtes partant de q et étiquetées par a sont stockées de façon contiguë et triés par ordre croissant dans le tableau "to_array", entre les indices intervals[q $*$ nb_labels + a] (inclus) et intervals[q $*$ nb_labels + a + 1] (exclus).

Remarques

> En particulier, la cellule intervals[nb_states $*$ nb_labels] contient le nombre total de transitions dans l'automate (la taille de intervals).

Définition à la ligne 52 du fichier graphs.h.

3.5.2 Documentation des champs

3.5.2.1 nb_states

```
int nb_states
```

Nombre de sommets.

Définition à la ligne 53 du fichier graphs.h.

3.5.2.2 nb_labels

```
int nb_labels
```

Taille de l'alphabet étiquetant les transitions.

Définition à la ligne 54 du fichier graphs.h.

3.5.2.3 to_array

```
int* to_array
```

Tableaux des transitions. Les transitions de la forme `(q,a,*)` sont consécutives dans to_array. Le début et la fin de la suite de ces transitions est donnée par le tableau intervals.

Définition à la ligne 55 du fichier graphs.h.

3.5.2.4 intervals

`int* intervals`

intervals[q* nb_letters + a] vaut l'indice du début de la zone des transitions (q,a,p) intervals[q* nb_letters
- a + 1] vaut l'indice de la fin (exclusif) de cette zone.

Définition à la ligne 59 du fichier graphs.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— graphs.h

3.6 Référence de la structure one_edge

Type utilisé pour représenter une unique arrête d'un graphe étiqueté.

`#include <graphs.h>`

Champs de données

— int from

État de départ.

— int to

État d'arrivée.

— int label

Étiquette de l'arrête.

3.6.1 Description détaillée

Type utilisé pour représenter une unique arrête d'un graphe étiqueté.

Définition à la ligne 74 du fichier graphs.h.

3.6.2 Documentation des champs

3.6.2.1 from

`int from`

État de départ.

Définition à la ligne 75 du fichier graphs.h.

3.6.2.2 to

`int to`

État d'arrivée.

Définition à la ligne 76 du fichier graphs.h.

3.6.2.3 label

`int label`

Étiquette de l'arrête.

Définition à la ligne 77 du fichier graphs.h.

La documentation de cette structure a été générée à partir du fichier suivant :
— graphs.h

3.7 Référence de la structure regexp

Type utilisé pour représenter un noeud d'une expression régulière.

`#include <regexp.h>`

Champs de données

— regelem op

L'opérateur utilisé.

— struct regexp * left

La sous-expression gauche (si il y en a une).

— struct regexp * right

La sous-expression droite (si il y en a une).

— char letter

La lettre (si il s'agit d'une lettre).

3.7.1 Description détaillée

Type utilisé pour représenter un noeud d'une expression régulière.

Les champs left et right sont utilisés pour représenter les sous-expressions gauche et droite (si elles existent):

- Pour les opérateurs unaire (COMPLEMENT et STAR), seul le champ left est utilisé.
- Pour les opérateurs binaires (UNION, INTER et CONCAT), les deux champs sont utilisés.
- Pour les opérandes (EMPTY, EPSILON et CHAR), aucun des deux champs n'est utilisé. Quand un champ n'est pas utilisé, il doit avoir la valeur NULL.

Définition à la ligne 52 du fichier regexp.h.

3.7.2 Documentation des champs

3.7.2.1 op

```
regelem op
```

L'opérateur utilisé.

Définition à la ligne 54 du fichier regexp.h.

3.7.2.2 left

```
struct regexp* left
```

La sous-expression gauche (si il y en a une).

Définition à la ligne 55 du fichier regexp.h.

3.7.2.3 right

```
struct regexp* right
```

La sous-expression droite (si il y en a une).

Définition à la ligne 56 du fichier regexp.h.

3.7.2.4 letter

```
char letter
```

La lettre (si il s'agit d'une lettre).

Définition à la ligne 57 du fichier regexp.h.

La documentation de cette structure a été générée à partir du fichier suivant :

— regexp.h

Chapitre 4

Documentation des fichiers

4.1 Référence du fichier alloc.c

Fonctions d'allocation mémoire.

```
#include "alloc.h"
```

Fonctions

— void multiple_free (void *p,...)

Fonction pour libérer de la mémoire en utilisant la fonction free().

— void check_null (const char *function, char *file, int line, int n,...)

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

4.1.1 Description détaillée

Fonctions d'allocation mémoire.

Définition dans le fichier alloc.c.

4.1.2 Documentation des fonctions

4.1.2.1 multiple_free()

```
void multiple_free (
           void * p,
            ...)
```

Fonction pour libérer de la mémoire en utilisant la fonction free().

Appelle free sur un nombre variable d'arguments. Le dernier argument doit être NULL.

Exemple d'utilisation :

```
int *p1, *p2, *p3;

CREATE_MEMORY(p1, 10); // Alloue un tableau de 10 int.
CREATE_MEMORY(p2, 10);
CREATE_MEMORY(p3, 10);
// ...
multiple_free(p1, p2, p3, NULL);
```

Attention

> Cette fonction est dangereuse et il faut savoir ce que vous faites si vous l'utilisez ! Les pointeurs à libérer **ne doivent pas** être NULL. Dans l'exemple précédent, si `p2` est NULL, on ne libère pas `p3` et cela peut créer une fuite mémoire.

Définition à la ligne 8 du fichier alloc.c.

4.1.2.2 check_null()

```
void check_null (
           const char * function,
           char * file,
           int line,
           int n,
            ...)
```

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

Voir également

> CHECK_NULL()

Définition à la ligne 18 du fichier alloc.c.

4.2 alloc.c

Aller à la documentation de ce fichier.

```
00001
00005
00006 #include "alloc.h"
00007
00008 void multiple_free(void *p, ...) {
00009   va_list ap;
00010   va_start(ap, p);
00011   while (p != NULL) {
00012     free(p);
00013     p = va_arg(ap, void *);
00014   }
00015   va_end(ap);
00016 }
00017
00018 void check_null(const char *function, char *file, int line, int n, ...) {
00019   va_list ap;
00020   char *s;
00021
00022   va_start(ap, n);
00023   for (int i = 0; i < n; i++) {
00024     void *p = va_arg(ap, void *);
00025     s = va_arg(ap, char *);
00026     if (p == NULL)
00027       PRINT_ERROR(KO, "CRITICAL", function, file, line, "%s is NULL.", s);
00028   }
00029 }
```

4.3 Référence du fichier alloc.h

Macros et fonctions d'allocation mémoire.

```
#include "error.h"
#include <stdarg.h>
#include <stdio.h>
#include <stdlib.h>
```

Macros

— #define CREATE_MEMORY(p, num_objects)

Macro pour allouer de la mémoire en utilisant la fonction malloc().

— #define CREATE_ZERO_MEMORY(p, num_objects)

Macro pour allouer de la mémoire en utilisant la fonction calloc().

— #define REDIMENSION_MEMORY(p, num_objects)

Macro pour réallouer de la mémoire en utilisant la fonction realloc().

— #define CHECK_NULL(n, ...)

Macro permettant de tester si un ou plusieurs pointeurs sont NULL.

Fonctions

— void check_null (const char *function, char *file, int line, int n,...)

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

— void multiple_free (void *p,...)

Fonction pour libérer de la mémoire en utilisant la fonction free().

4.3.1 Description détaillée

Macros et fonctions d'allocation mémoire.

Les macros CREATE_MEMORY(), CREATE_ZERO_MEMORY() et REDIMENSION_MEMORY() définies dans ce fichier permettent d'allouer de la mémoire dynamiquement, en vérifiant que l'allocation a réussi. En cas d'échec, elles affichent un message d'erreur sur la sortie erreur, en indiquant la fonction, le fichier et la ligne où l'allocation a échoué, et quittent le programme.

La fonction multiple_free() permet de libérer plusieurs pointeurs en un seul appel (mais attention, aucun de ces pointeurs ne doit être `NULL`, et tous doivent avoir été alloués dynamiquement avant cette libération).

La macro CHECK_NULL() permet de vérifier qu'un ou plusieurs pointeurs ne sont pas `NULL`.

La fonction check_null() n'est pas destinée à être utilisée directement, mais est utilisée par la macro CHECK_NULL().

Définition dans le fichier alloc.h.

4.3.2 Documentation des macros

4.3.2.1 CREATE_MEMORY

```
#define CREATE_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Allocating  %lu objects.", (unsigned long)(num_objects));    \
        void *tmp = malloc(sizeof *(p) * (num_objects));                    \
        if (!tmp)                                                           \
            FATAL("Malloc error.");                                         \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour allouer de la mémoire en utilisant la fonction malloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nombre d'objets à allouer.

Alloue de la mémoire permettant de stocker `num_objects`, chacun de type `*p`.

Exemple d'utilisation :

```
backmove *p;
CREATE_MEMORY(p, 10); // Alloue un tableau de 10 backmove.
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

> Le pointeur p doit avoir été déclaré, mais ne pas pointer vers une zone déjà allouée dynamiquement (sous peine de créer une fuite mémoire). Si p pointe déjà vers une zone allouée dynamiquement, il faut utiliser REDIMENSION_MEMORY.

Voir également

> REDIMENSION_MEMORY()

Définition à la ligne 57 du fichier alloc.h.

4.3.2.2 CREATE_ZERO_MEMORY

```
#define CREATE_ZERO_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Callocating  %lu objects.", (unsigned long)(num_objects));   \
        void *tmp = calloc((num_objects), sizeof *(p));                     \
        if (!tmp)                                                           \
            FATAL("Calloc error.");                                         \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour allouer de la mémoire en utilisant la fonction calloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nombre d'objets à allouer.

Alloue de la mémoire initialisée à 0, permettant de stocker `num_objects`, chacun de type `*p`.

Exemple d'utilisation :

```
backmove *p;
CREATE_ZERO_MEMORY(p, 10); // Alloue un tableau de 10 backmove initialisé à 0
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

> Le pointeur `p` doit avoir été déclaré, mais ne pas pointer vers une zone déjà allouée dynamiquement (sous peine de créer une fuite mémoire). Si `p` pointe déjà vers une zone allouée dynamiquement, il faut utiliser REDIMENSION_MEMORY.

Voir également

> REDIMENSION_MEMORY()

Définition à la ligne 92 du fichier alloc.h.

4.3.2.3 REDIMENSION_MEMORY

```
#define REDIMENSION_MEMORY(
            p,
            num_objects)
```

Valeur :

```
    do {                                                                    \
        DEBUG("Reallocating  %lu objects.", (unsigned long)(num_objects));  \
        void *tmp = realloc((p), sizeof *(p) * (num_objects));              \
        if (!tmp) {                                                         \
            FATAL("Realloc error.");                                        \
        }                                                                   \
        (p) = tmp;                                                          \
    } while (0)
```

Macro pour réallouer de la mémoire en utilisant la fonction realloc().

Paramètres

p	Pointeur à allouer.
num_objects	Nouveau nombre d'objets à allouer.

Ré-alloue la mémoire pointée par un pointeur p, dont la valeur doit avoir été retournée précédemment par une fonction d'allocation dynamique (malloc, calloc, realloc ou une des macros de ce fichier).

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Exemple d'utilisation :

```
backmove *p;
CREATE_MEMORY(p, 10); // Alloue un tableau de 10 backmove.
                      // ...
REDIMENSION_MEMORY(p, 20); // Ré-alloue un tableau de 20 backmove en copiant
                           // le tableau précédent en début ce nouveau
                           // tableau, et affecte à p l'adresse de la
                           // première case de ce tableau.
```

Si l'allocation échoue, affiche un message d'erreur et quitte le programme.

Attention

Le pointeur p doit avoir été déclaré et pointer vers une zone déjà allouée dynamiquement.

Voir également

CREATE_MEMORY(), CREATE_ZERO_MEMORY()

Définition à la ligne 132 du fichier alloc.h.

4.3.2.4 CHECK_NULL

```
#define CHECK_NULL(
            n,
            ...)
```

Valeur :

```
check_null(__func__, __FILE__, __LINE__, n __VA_OPT__(, ) __VA_ARGS__)
```

Macro permettant de tester si un ou plusieurs pointeurs sont NULL.

Paramètres

n	Nombre de pointeurs à tester.
...	Pointeurs à tester, en alternance avec des chaînes de caractères.

Il doit y avoir un pointeur à tester pour chaque chaîne de caractères, et inversement.

La chaîne de caractères qui suit chaque pointeur permet de personnaliser le message d'erreur.

Exemple d'utilisation :

```
CHECK_NULL(3, p1, "Name1", p2, "Name2", p3, "Name3");
```

Ici, 3 est le nombre de pointeurs à vérifier, et `p1`, `p2` et `p3` sont les pointeurs. Si `p1` et `p3` sont NULL, la macro affichera un message d'erreur de la forme :

[CRITICAL] Name1 is NULL!
[CRITICAL] Name3 is NULL!

Voir également

check_null()

Définition à la ligne 168 du fichier alloc.h.

4.3.3 Documentation des fonctions

4.3.3.1 check_null()

```
void check_null (
            const char * function,
            char * file,
            int line,
            int n,
             ...)
```

Ne pas utiliser cette fonction directement, mais à la place la macro CHECK_NULL(), qui indique dans le message la position de l'erreur dans le source.

Voir également

CHECK_NULL()

Définition à la ligne 18 du fichier alloc.c.

4.3.3.2 multiple_free()

```
void multiple_free (
           void * p,
            ...)
```

Fonction pour libérer de la mémoire en utilisant la fonction free().

Appelle free sur un nombre variable d'arguments. Le dernier argument doit être NULL.

Exemple d'utilisation :

```
int *p1, *p2, *p3;

CREATE_MEMORY(p1, 10); // Alloue un tableau de 10 int.
CREATE_MEMORY(p2, 10);
CREATE_MEMORY(p3, 10);
// ...
multiple_free(p1, p2, p3, NULL);
```

Attention

> Cette fonction est dangereuse et il faut savoir ce que vous faites si vous l'utilisez ! Les pointeurs à libérer **ne doivent pas** être NULL. Dans l'exemple précédent, si p2 est NULL, on ne libère pas p3 et cela peut créer une fuite mémoire.

Définition à la ligne 8 du fichier alloc.c.

4.4 alloc.h

Aller à la documentation de ce fichier.

```
00001
00022
00023 #ifndef ALLOC_H_
00024 #define ALLOC_H_
00025
00026 #include "error.h"
00027 #include <stdarg.h>
00028 #include <stdio.h>
00029 #include <stdlib.h>
00030
00056
00057 #define CREATE_MEMORY(p, num_objects)                                    \
00058     do {                                                                 \
00059         DEBUG("Allocating  %lu objects.", (unsigned long)(num_objects)); \
00060         void *tmp = malloc(sizeof *(p) * (num_objects));                 \
00061         if (!tmp)                                                        \
00062             FATAL("Malloc error.");                                      \
00063         (p) = tmp;                                                       \
00064     } while (0)
00065
00091
```

```
00092 #define CREATE_ZERO_MEMORY(p, num_objects)                                   \
00093     do {                                                                      \
00094         DEBUG("Callocating  %lu objects.", (unsigned long)(num_objects));     \
00095         void *tmp = calloc((num_objects), sizeof *(p));                       \
00096         if (!tmp)                                                             \
00097             FATAL("Calloc error.");                                           \
00098         (p) = tmp;                                                            \
00099     } while (0)
00100
00131
00132 #define REDIMENSION_MEMORY(p, num_objects)                                   \
00133     do {                                                                      \
00134         DEBUG("Reallocating  %lu objects.", (unsigned long)(num_objects));    \
00135         void *tmp = realloc((p), sizeof *(p) * (num_objects));                \
00136         if (!tmp) {                                                           \
00137             FATAL("Realloc error.");                                          \
00138         }                                                                     \
00139         (p) = tmp;                                                            \
00140     } while (0)
00141
00167
00168 #define CHECK_NULL(n, ...)                                                   \
00169     check_null(__func__, __FILE__, __LINE__, n __VA_OPT__(, ) __VA_ARGS__)
00170
00178
00179 void check_null(const char *function, char *file, int line, int n, ...);
00180
00203
00204 void multiple_free(void *p, ...);
00205
00206 #endif // ALLOC_H_
```

4.5 Référence du fichier error.h

Macros permettant d'afficher des messages d'erreur.

Macros

— #define ATTENTION "\xe2\x9d\x97"

Symbole ❗.

— #define HINT "\xe2\x9c\xa8"

Symbole ✨.

— #define OK "\xE2\x9C\x85"

Symbole ✅.

— #define KO "\xE2\x9D\x8C"

Symbole ❌.

— #define DEBUG_LEVEL INFO

Niveau de debug, INFO par défaut.

— #define PRINT_ERROR(symbol, name, function, file, line, msg, ...)

Affiche un message d'erreur sur la sortie d'erreur.

— #define FATAL(msg, ...)

Affiche un message de niveau FATAL et termine le programme.

— #define PRINT_DEBUG(symbol, name, level, msg, ...)

— #define TRACE(msg, ...)

Affiche un message de niveau TRACE.

— #define DEBUG(msg, ...)

Affiche un message de niveau DEBUG.

— #define INFO(msg, ...)

Affiche un message de niveau INFO.

— #define WARNING(msg, ...)

Affiche un message de niveau WARNING.

— #define ERROR(msg, ...)

Affiche un message de niveau ERROR.

— #define CRITICAL(msg, ...)

Affiche un message de niveau CRITICAL.

Énumérations

— enum debug_level {
OFF , CRITICAL , ERROR , WARNING ,
INFO , DEBUG , TRACE }

— enum msg_level {
CRITICAL_L , ERROR_L , WARNING_L , INFO_L ,
DEBUG_L , TRACE_L }

4.5.1 Description détaillée

Macros permettant d'afficher des messages d'erreur.

Le fichier fournit les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL(), qui permettent d'afficher des messages d'erreur sur la sortie d'erreur, et, pour la macro FATAL(), de terminer le programme. Les messages d'erreur sont affichés avec le nom de la fonction, le nom du fichier et le numéro de la ligne où l'appel à la macro est effectué.

Les autres macros du fichier sont essentiellement destinées à implémenter les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL(). Elles ne sont pas destinées à être utilisées directement par l'utilisateur (mais rien ne l'empêche).

Les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL() prennent en argument un message à afficher, qui peut contenir des spécificateurs de conversion comme dans la fonction printf. Elles peuvent prendre des arguments supplémentaires, un par spécificateur de conversion.

Exemple d'utilisation :

```
#include "error.h"
ERROR("La variable colors vaut %d, elle devrait être supérieure à %d.",
       colors, MIN_COLORS);
```

Note

Les messages d'un certain type sont affichés en fonction du niveau de debug, fixé par la variable DEBUG_LEVEL. Par exemple, si le niveau de debug est à ERROR, seules les macros ERROR(), CRITICAL() et FATAL() afficheront leur message. Les autres macros n'afficheront rien. Par défaut, le niveau de debug est INFO.

Niveaux de debug :

- OFF (seulement les messages FATAL sont affichés),
- CRITICAL (seulement les messages CRITICAL et FATAL),
- ERROR (seulement les messages ERROR, CRITICAL et FATAL),
- WARNING (seulement les messages WARNING, ERROR, CRITICAL et FATAL),
- INFO (tous les messages sauf DEBUG et TRACE),
- DEBUG (tous les messages sauf TRACE),
- TRACE (tous les messages).

Pour changer le niveau de debug, il faut définir la macro DEBUG_LEVEL avant d'inclure error.h, typiquement à la compilation. Par exemple, pour afficher tous les messages de debug, il faut définir DEBUG_LEVEL à TRACE. Pour cela, on peut compiler le programme ainsi :

```
make -B CFLAGS="-DDEBUG_LEVEL=TRACE"
```

Voir également

DEBUG_LEVEL et les macros TRACE(), DEBUG(), INFO(), WARNING(), ERROR(), CRITICAL() et FATAL().

Définition dans le fichier error.h.

4.5.2 Documentation des macros

4.5.2.1 ATTENTION

```
#define ATTENTION "\xe2\x9d\x97"
```

Symbole ❗.

Définition à la ligne 60 du fichier error.h.

4.5.2.2 HINT

```
#define HINT "\xe2\x9c\xa8"
```

Symbole ✨.

Définition à la ligne 62 du fichier error.h.

4.5.2.3 OK

```
#define OK "\xE2\x9C\x85"
```

Symbole ✅.

Définition à la ligne 64 du fichier error.h.

4.5.2.4 KO

```
#define KO "\xE2\x9D\x8C"
```

Symbole ❌.

Définition à la ligne 66 du fichier error.h.

4.5.2.5 DEBUG_LEVEL

```
#define DEBUG_LEVEL INFO
```

Niveau de debug, INFO par défaut.

Par défaut, le niveau de debug est INFO. Pour changer le niveau de debug, il faut définir la macro DEBUG_LEVEL avant d'inclure error.h.

Cela peut se faire au moment de la compilation :

```
make -B CFLAGS="-DDEBUG_LEVEL=WARNING"
```

Par exemple, pour afficher tous les messages de debug, il faut définir DEBUG_LEVEL à TRACE. Pour n'afficher que les messages de niveau FATAL, il faut définir DEBUG_LEVEL à OFF.

Niveaux possibles :

- OFF (seulement les messages FATAL sont affichés),
- CRITICAL (seulement les messages CRITICAL et FATAL),
- ERROR (seulement les messages ERROR, CRITICAL et FATAL),
- WARNING (seulement les messages WARNING, ERROR, CRITICAL et FATAL),
- INFO (tous les messages sauf DEBUG et TRACE),
- DEBUG (tous les messages sauf TRACE),
- TRACE (tous les messages).

Voir également

enum debug_level

Définition à la ligne 103 du fichier error.h.

4.5.2.6 PRINT_ERROR

```
#define PRINT_ERROR(
            symbol,
            name,
            function,
            file,
            line,
            msg,
            ...)
```

Valeur :

```
  do {                                                    \
    fprintf(stderr,                                       \
            "\n" symbol                                   \
            " ["                                          \
            name                                          \
            "]"                                           \
            " - Function %s (%s:%d) -\n  "                \
```

```
            msg "\n",                                                   \
            function, file, line                                        \
            __VA_OPT__( , ) __VA_ARGS__);                               \
} while (0)
```

Affiche un message d'erreur sur la sortie d'erreur.

Cette macro est utilisée par les macros TRACE, DEBUG, INFO, WARNING, ERROR, CRITICAL et FATAL.

Paramètres

symbol	Symbole à afficher (ATTENTION, HINT, OK ou KO).
name	Nom du message (comme "TRACE", "DEBUG", "INFO", "WARNING", "ERROR", "↩ CRITICAL" ou "FATAL").
function	Nom de la fonction où le message est affiché.
file	Nom du fichier où le message est affiché.
line	Numéro de la ligne où le message est affiché.
msg	Message à afficher. Ce message peut contenir les mêmes spécificateurs de conversion que la fonction printf (comme `d`, `s`, etc.).
...	Arguments supplémentaires à afficher avec le message, un par spécificateur de conversion.

Définition à la ligne 125 du fichier error.h.

4.5.2.7 FATAL

```
#define FATAL(
            msg,
            ...)
```

Valeur :

```
do {                                                    \
  PRINT_ERROR(KO, "FATAL", __func__, __FILE__, __LINE__, \
              msg __VA_OPT__( , ) __VA_ARGS__);         \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Affiche un message de niveau FATAL et termine le programme.

Définition à la ligne 140 du fichier error.h.

4.5.2.8 PRINT_DEBUG

```
#define PRINT_DEBUG(
            symbol,
            name,
            level,
            msg,
            ...)
```

Valeur :

```
do {                                                      \
  if ((int)DEBUG_LEVEL > (int)level)                      \
  PRINT_ERROR(symbol, name, __func__, __FILE__, __LINE__, \
              msg __VA_OPT__( , ) __VA_ARGS__);           \
} while (0)
```

Définition à la ligne 147 du fichier error.h.

4.5.2.9 TRACE

```
#define TRACE(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(OK, "TRACE", TRACE_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau TRACE.

Définition à la ligne 155 du fichier error.h.

4.5.2.10 DEBUG

```
#define DEBUG(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(HINT, "DEBUG", DEBUG_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau DEBUG.

Définition à la ligne 160 du fichier error.h.

4.5.2.11 INFO

```
#define INFO(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(HINT, "INFO", INFO_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau INFO.

Définition à la ligne 165 du fichier error.h.

4.5.2.12 WARNING

```
#define WARNING(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(ATTENTION, "WARNING", WARNING_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau WARNING.

Définition à la ligne 170 du fichier error.h.

4.5.2.13 ERROR

```
#define ERROR(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(ATTENTION, "ERROR", ERROR_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau ERROR.

Définition à la ligne 175 du fichier error.h.

4.5.2.14 CRITICAL

```
#define CRITICAL(
            msg,
            ...)
```

Valeur :

```
PRINT_DEBUG(KO, "CRITICAL", CRITICAL_L, \
            msg, __VA_ARGS__)
```

Affiche un message de niveau CRITICAL.

Définition à la ligne 180 du fichier error.h.

4.5.3 Documentation du type de l'énumération

4.5.3.1 debug_level

```
enum debug_level
```

Valeurs énumérées

OFF	
CRITICAL	
ERROR	
WARNING	
INFO	
DEBUG	
TRACE	

Définition à la ligne 70 du fichier error.h.

4.5.3.2 msg_level

enum msg_level

Valeurs énumérées

CRITICAL↩ _L	
ERROR_L	
WARNING↩ _L	
INFO_L	
DEBUG_L	
TRACE_L	

Définition à la ligne 71 du fichier error.h.

4.6 error.h

Aller à la documentation de ce fichier.

```
00001
00054
00055 #ifndef ERROR_H_
00056 #define ERROR_H_
00057
00058
00060 #define ATTENTION "\xe2\x9d\x97"
```

```
00062 #define HINT "\xe2\x9c\xa8"
00064 #define OK "\xE2\x9C\x85"
00066 #define KO "\xE2\x9D\x8C"
00067
00068 // In X mode, all messages more severe than X are printed.
00069 // clang-format off
00070 enum debug_level {OFF,        CRITICAL, ERROR,     WARNING, INFO,    DEBUG,
      TRACE};
00071 enum msg_level   {CRITICAL_L, ERROR_L,  WARNING_L, INFO_L,  DEBUG_L, TRACE_L};
00072 // clang-format on
00073
00101
00102 #ifndef DEBUG_LEVEL
00103 #define DEBUG_LEVEL INFO
00104 #endif
00105
00106 // __VA_OPTS__ requires C2x. Clang issues spurious warning.
00107
00124
00125 #define PRINT_ERROR(symbol, name, function, file, line, msg, ...) \
00126   do {                                                            \
00127     fprintf(stderr,                                               \
00128             "\n" symbol                                           \
00129             " ["                                                  \
00130             name                                                  \
00131             "]"                                                   \
00132             " - Function %s (%s:%d) -\n  "                        \
00133             msg "\n",                                             \
00134             function, file, line                                  \
00135             __VA_OPT__( , ) __VA_ARGS__);                         \
00136   } while (0)
00137
00139
00140 #define FATAL(msg, ...)                                    \
00141   do {                                                     \
00142     PRINT_ERROR(KO, "FATAL", __func__, __FILE__, __LINE__, \
00143                 msg __VA_OPT__( , ) __VA_ARGS__);          \
00144     exit(EXIT_FAILURE);                                    \
00145   } while (0)
00146
00147 #define PRINT_DEBUG(symbol, name, level, msg, ...)          \
00148   do {                                                      \
00149     if ((int)DEBUG_LEVEL > (int)level)                      \
00150     PRINT_ERROR(symbol, name, __func__, __FILE__, __LINE__, \
00151                 msg __VA_OPT__( , ) __VA_ARGS__);           \
00152   } while (0)
00153
00155 #define TRACE(msg, ...)             \
00156   PRINT_DEBUG(OK, "TRACE", TRACE_L, \
00157               msg, __VA_ARGS__)
00158
00160 #define DEBUG(msg, ...)               \
00161   PRINT_DEBUG(HINT, "DEBUG", DEBUG_L, \
00162               msg, __VA_ARGS__)
00163
00165 #define INFO(msg, ...)              \
00166   PRINT_DEBUG(HINT, "INFO", INFO_L, \
00167               msg, __VA_ARGS__)
```

```
00168
00170 #define WARNING(msg, ...)                        \
00171   PRINT_DEBUG(ATTENTION, "WARNING", WARNING_L, \
00172               msg, __VA_ARGS__)
00173
00175 #define ERROR(msg, ...)                       \
00176   PRINT_DEBUG(ATTENTION, "ERROR", ERROR_L, \
00177               msg, __VA_ARGS__)
00178
00180 #define CRITICAL(msg, ...)                  \
00181   PRINT_DEBUG(KO, "CRITICAL", CRITICAL_L, \
00182               msg, __VA_ARGS__)
00183
00184 #endif // ERROR_H_
```

4.7 Référence du fichier graphs.c

```
#include "graphs.h"
```

Fonctions

— one_edge * ngraph_to_edge_list (ngraph *, int *size)

Conversion d'un graphe en une liste d'arrêtes.

— ngraph * edge_list_to_ngraph (one_edge *, int, int, int)

Conversion d'une liste d'arrêtes en un graphe étiqueté.

— multi_edge * ngraph_to_multi_edges (ngraph *, int *)

Calcul de la liste des multi-arêtes d'un graphe.

4.7.1 Documentation des fonctions

4.7.1.1 ngraph_to_edge_list()

```
one_edge * ngraph_to_edge_list (
           ngraph * G,
           int * size)
```

Conversion d'un graphe en une liste d'arrêtes.

Remarques

La taille de la liste est retournée par le second paramètre.

Attention

Cette fonction est utilisée pour la sauvegarde d'un NFA dans un fichier. Elle doit être écrite pour que la sauvegarde fonctionne correctement.

Renvoie

La liste des arrêtes.

Paramètres

G	Le graphe à convertir.
size	Pointeur pour retourner la taille de la liste.

Définition à la ligne 3 du fichier graphs.c.

4.7.1.2 edge_list_to_ngraph()

```
ngraph * edge_list_to_ngraph (
            one_edge * edges,
            int size,
            int nb_states,
            int nb_labels)
```

Conversion d'une liste d'arrêtes en un graphe étiqueté.

Attention

Cette fonction est utilisée pour le chargement d'un NFA depuis un fichier. Elle est aussi utilisée par l'algorithme de Glushkov. Elle doit être écrite pour que ces deux fonctionnalités fonctionnent correctement.

Renvoie

Le graphe étiqueté.

Paramètres

edges	La liste des arrêtes.
size	La taille de la liste.
nb_states	Le nombre total d'états du graphe.

nb_labels	Le nombre total d'étiquettes du graphe.

Définition à la ligne 12 du fichier graphs.c.

4.7.1.3 ngraph_to_multi_edges()

```
multi_edge * ngraph_to_multi_edges (
            ngraph * G,
            int * size)
```

Calcul de la liste des multi-arêtes d'un graphe.

Renvoie

La liste des multi-arêtes stockée dans une pile.

Remarques

Cette fonction sert à afficher les automates de façon plus compacte : une fois écrite, les arêtes qui ont les mêmes extrémités peuvent être regroupées en une seule flèche. Pour utiliser cette fonction, il faut basculer à 1 la macro ADVANCED de parameters.h.

Paramètres

G	Le graphe,
size	Pointeur pour retourner la taille de la liste.

Définition à la ligne 18 du fichier graphs.c.

4.8 graphs.c

Aller à la documentation de ce fichier.

```
00001 #include "graphs.h"
00002
00003 one_edge *ngraph_to_edge_list(ngraph *, int *size)
00004 {
00005     if (size)
00006     {
00007         *size = 0;
00008     }
00009     return NULL;
00010 }
00011
00012 ngraph *edge_list_to_ngraph(one_edge *, int, int, int)
```

```
00013 {
00014
00015     return NULL;
00016 }
00017
00018 multi_edge *ngraph_to_multi_edges(ngraph *, int *)
00019 {
00020     return NULL;
00021 }
```

4.9 Référence du fichier graphs.h

Implémentation des graphes étiquetés.

```
#include "type_basic.h"
#include "type_boolarray.h"
#include <stdarg.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
```

Structures de données

— struct ngraph

Type utilisé pour représenter un graphe orienté et étiqueté.

— struct one_edge

Type utilisé pour représenter une unique arrête d'un graphe étiqueté.

— struct multi_edge

Type utilisé pour représenter une multi-arête.

Définitions de type

— typedef struct ngraph ngraph

Type utilisé pour représenter un graphe orienté et étiqueté.

Fonctions

— one_edge * ngraph_to_edge_list (ngraph *G, int *size)

Conversion d'un graphe en une liste d'arrêtes.

— ngraph * edge_list_to_ngraph (one_edge *edges, int size, int nb_states, int nb_labels)

Conversion d'une liste d'arrêtes en un graphe étiqueté.

— multi_edge * ngraph_to_multi_edges (ngraph *G, int *size)

Calcul de la liste des multi-arêtes d'un graphe.

4.9.1 Description détaillée

Implémentation des graphes étiquetés.

Contient le type utilisé pour représenter des graphes étiquetés (qu'on utilisera ensuite pour coder les transitions des automates).

Définition dans le fichier graphs.h.

4.9.2 Documentation des définitions de type

4.9.2.1 ngraph

```
typedef struct ngraph ngraph
```

Type utilisé pour représenter un graphe orienté et étiqueté.

Le graphe contient nb_states sommets, numérotés de 0 à nb_states-1 et il y a nb_labels étiquettes numérotées de 0 à nb_labels-1. Les arrêtes sont codées par les deux tableaux "to_array" et "intervals":

- "intervals" est un tableau de taille "nb_states * nb_labels + 1". Il stocke des indices dans le tableau "to_array" triés par ordre croissant.
- "to_array" est un tableau d'entiers, qui stocke les états d'arrivée des arrêtes (sa taille est égale au nombre total d'arrêtes dans le graphe). Pour chaque état q et chaque étiquette a, les états d'arrivée des arrêtes partant de q et étiquetées par a sont stockées de façon contiguë et triés par ordre croissant dans le tableau "to_array", entre les indices intervals[q * nb_labels + a] (inclus) et intervals[q * nb_labels + a + 1] (exclus).

Remarques

En particulier, la cellule intervals[nb_states * nb_labels] contient le nombre total de transitions dans l'automate (la taille de intervals).

4.9.3 Documentation des fonctions

4.9.3.1 ngraph_to_edge_list()

```
one_edge * ngraph_to_edge_list (
           ngraph * G,
           int * size)
```

Conversion d'un graphe en une liste d'arrêtes.

Remarques

La taille de la liste est retournée par le second paramètre.

Attention

Cette fonction est utilisée pour la sauvegarde d'un NFA dans un fichier. Elle doit être écrite pour que la sauvegarde fonctionne correctement.

Renvoie

La liste des arrêtes.

Paramètres

G	Le graphe à convertir.
size	Pointeur pour retourner la taille de la liste.

Définition à la ligne 3 du fichier graphs.c.

4.9.3.2 edge_list_to_ngraph()

```
ngraph * edge_list_to_ngraph (
           one_edge * edges,
           int size,
           int nb_states,
           int nb_labels)
```

Conversion d'une liste d'arrêtes en un graphe étiqueté.

Attention

Cette fonction est utilisée pour le chargement d'un NFA depuis un fichier. Elle est aussi utilisée par l'algorithme de Glushkov. Elle doit être écrite pour que ces deux fonctionnalités fonctionnent correctement.

Renvoie

Le graphe étiqueté.

Paramètres

edges	La liste des arrêtes.
size	La taille de la liste.
nb_states	Le nombre total d'états du graphe.
nb_labels	Le nombre total d'étiquettes du graphe.

Définition à la ligne 12 du fichier graphs.c.

4.9.3.3 ngraph_to_multi_edges()

```
multi_edge * ngraph_to_multi_edges (
           ngraph * G,
           int * size)
```

Calcul de la liste des multi-arêtes d'un graphe.

Renvoie

La liste des multi-arêtes stockée dans une pile.

Remarques

Cette fonction sert à afficher les automates de façon plus compacte : une fois écrite, les arêtes qui ont les mêmes extrémités peuvent être regroupées en une seule flèche. Pour utiliser cette fonction, il faut basculer à 1 la macro ADVANCED de parameters.h.

Paramètres

G	Le graphe,
size	Pointeur pour retourner la taille de la liste.

Définition à la ligne 18 du fichier graphs.c.

4.10 graphs.h

Aller à la documentation de ce fichier.

```
00001
00010
00011 #ifndef GRAPHS_H
00012 #define GRAPHS_H
00013
00014 /*   ____ ____      _    ____  _   _ _____ ____    */
00015 /*  / ___|  _ \    / \  |  _ \| | | | ____/ ___|  */
```

```
00016 /* | |  _| |_) |  / _ \ | |_) | |_| |  _| \___ \  */
00017 /* | |_| |  _ <  / ___ \|  __/|  _  | |___ ___) | */
00018 /*  \____|_| \_\/_/   \_\_|   |_| |_|_____|____/  */
00019
00020 #include "type_basic.h"
00021 #include "type_boolarray.h"
00022 #include <stdarg.h>
00023 #include <stdbool.h>
00024 #include <stdio.h>
00025 #include <stdlib.h>
00026
00027 /*****************************************************************/
00028 /*+ Représentations par des tableaux (type utilisé dans les NFAs) +*/
00029 /*****************************************************************/
00030
00052 typedef struct ngraph {
00053     int nb_states;
00054     int nb_labels;
00055     int *to_array;
00059     int *intervals;
00063 } ngraph;
00064
00065 /*****************************************/
00066 /*+ Représentation par une liste d'arrêtes +*/
00067 /*****************************************/
00068
00074 typedef struct {
00075     int from;
00076     int to;
00077     int label;
00078 } one_edge;
00079
00094 one_edge *ngraph_to_edge_list(
00095     ngraph *G,
00096     int *size
00097 );
00098
00111 ngraph *
00112 edge_list_to_ngraph(one_edge *edges,
00113                     int size,
00114                     int nb_states,
00115                     int nb_labels
00116 );
00117
00133 typedef struct {
00134     int in;
00135     barray *lab;
00136     int out;
00137 } multi_edge;
00138
00151 multi_edge *ngraph_to_multi_edges(
00152     ngraph *G,
00153     int *size
00154 );
00155
00156 #endif
```

4.11 Référence du fichier grep.c

```
#include "grep.h"
```

Fonctions

— void grep (const char *, const char *)

Affiche les lignes d'un fichier contenant un motif correspondant à une expression régulière.

4.11.1 Documentation des fonctions

4.11.1.1 grep()

```
void grep (
            const char * regexp,
            const char * filename)
```

Affiche les lignes d'un fichier contenant un motif correspondant à une expression régulière.

Renvoie

Une copie du tableau des noms de lettres

Paramètres

regexp	L'expression régulière donnant le motif à chercher.
filename	Le fichier dans lequel rechercher le motif.

Définition à la ligne 3 du fichier grep.c.

4.12 grep.c

Aller à la documentation de ce fichier.

```
00001 #include "grep.h"
00002
00003 void grep(const char *, const char *) {
00004     return;
00005 }
```

4.13 Référence du fichier grep.h

Implémentation de la fonction grep.

```
#include "parse_regexp.h"
#include "regexp.h"
#include <stdio.h>
```

Fonctions

— void grep (const char *regexp, const char *filename)

Affiche les lignes d'un fichier contenant un motif correspondant à une expression régulière.

4.13.1 Description détaillée

Implémentation de la fonction grep.

Contient une unique fonction, utilisée pour rechercher toutes les lignes d'un fichier qui contiennent un motif décrit par une expression régulière.

Définition dans le fichier grep.h.

4.13.2 Documentation des fonctions

4.13.2.1 grep()

```
void grep (
           const char * regexp,
           const char * filename)
```

Affiche les lignes d'un fichier contenant un motif correspondant à une expression régulière.

Renvoie

Une copie du tableau des noms de lettres

Paramètres

regexp	L'expression régulière donnant le motif à chercher.
filename	Le fichier dans lequel rechercher le motif.

Définition à la ligne 3 du fichier grep.c.

4.14 grep.h

Aller à la documentation de ce fichier.

```
00001
00012
00013 #ifndef GREP_H_
00014 #define GREP_H_
00015
00016 #include "parse_regexp.h"
00017 #include "regexp.h"
00018 #include <stdio.h>
00019
00028
00029 void grep(
00030     const char *regexp,
00031     const char *filename
00032 );
00033
00034 #endif // GREP_H_
```

4.15 Référence du fichier nfa.c

```
#include "nfa.h"
#include "printing.h"
```

Fonctions

— void nfa_reset_state_names (nfa *A)

Efface les noms des états d'un NFA (si il y en a).

— nfa * create_emptylang (void)

Crée un NFA qui reconnaît le langage vide.

— nfa * create_sing_epsilon (void)

Créé un NFA qui reconnaît le singleton {epsilon}.

— nfa * create_sing_letter (char)

Créé un NFA qui reconnaît un singleton contenant un mot d'une seule lettre.

— void nfa_delete (nfa *)

Suppression d'un NFA.

— nfa * nfa_copy (nfa *)

Création d'une copie d'un NFA.

— nfa * nfa_union (nfa *, nfa *)

Union de deux NFAs.

— nfa * nfa_concat (nfa *, nfa *)

Concaténation de deux NFAs.

— nfa * nfa_star (nfa *)

Étoile de Kleene d'un NFA.

— nfa * nfa_trim (nfa *)

Élimination des états inutiles.

— nfa * nfa_mirror (nfa *)

Miroir d'un NFA.

— nfa * nfa_elimeps (nfa *)

Élimination des transitions epsilon d'un NFA.

— barray * nfa_compute_runs (nfa *, const char *)

Calcule l'ensemble des états qui sont atteints par un mot dans un NFA.

4.15.1 Documentation des fonctions

4.15.1.1 nfa_reset_state_names()

```
void nfa_reset_state_names (
          nfa * A)
```

Efface les noms des états d'un NFA (si il y en a).

Définition à la ligne 4 du fichier nfa.c.

4.15.1.2 create_emptylang()

```
nfa * create_emptylang (
          void )
```

Crée un NFA qui reconnaît le langage vide.

Remarques

L'alphabet du NFA est vide également.

Renvoie

Le NFA.

Définition à la ligne 24 du fichier nfa.c.

4.15.1.3 create_sing_epsilon()

```
nfa * create_sing_epsilon (
          void )
```

Créé un NFA qui reconnaît le singleton {epsilon}.

Remarques

L'alphabet du NFA est vide.

Renvoie

Le NFA.

Définition à la ligne 29 du fichier nfa.c.

4.15.1.4 create_sing_letter()

```
nfa * create_sing_letter (
          char letter)
```

Créé un NFA qui reconnaît un singleton contenant un mot d'une seule lettre.

Remarques

L'alphabet du NFA contient une seule lettre.

Renvoie

Le NFA.

Définition à la ligne 34 du fichier nfa.c.

4.15.1.5 nfa_delete()

```
void nfa_delete (
            nfa * A)
```

Suppression d'un NFA.

Définition à la ligne 39 du fichier nfa.c.

4.15.1.6 nfa_copy()

```
nfa * nfa_copy (
            nfa * A)
```

Création d'une copie d'un NFA.

Renvoie

La copie.

Définition à la ligne 48 du fichier nfa.c.

4.15.1.7 nfa_union()

```
nfa * nfa_union (
            nfa * A1,
            nfa * A2)
```

Union de deux NFAs.

Attention

Les alphabets des deux NFAs peuvent être distincts. Dans ce cas, il faudra les étendre à l'union des deux alphabets.

Renvoie

Un nouveau NFA qui réalise l'union.

Définition à la ligne 53 du fichier nfa.c.

4.15.1.8 nfa_concat()

```
nfa * nfa_concat (
          nfa * A1,
          nfa * A2)
```

Concaténation de deux NFAs.

Attention

Les alphabets des deux NFAs peuvent être distincts. Dans ce cas, il faudra les étendre à l'union des deux alphabets.

Renvoie

Un nouveau NFA qui réalise la concaténation.

Définition à la ligne 58 du fichier nfa.c.

4.15.1.9 nfa_star()

```
nfa * nfa_star (
          nfa * A)
```

Étoile de Kleene d'un NFA.

Renvoie

Un nouveau NFA qui réalise l'étoile.

Définition à la ligne 63 du fichier nfa.c.

4.15.1.10 nfa_trim()

```
nfa * nfa_trim (
          nfa * A)
```

Élimination des états inutiles.

Remarques

Est inutile tout état inaccessible depuis un état initial ou depuis lequel on ne peut pas atteindre un état final.

Renvoie

Un nouveau NFA dans lequel on a éliminé les états inutiles.

Définition à la ligne 68 du fichier nfa.c.

4.15.1.11 nfa_mirror()

```
nfa * nfa_mirror (
          nfa * A)
```

Miroir d'un NFA.

Renvoie

Un nouveau NFA qui réalise le miroir.

Définition à la ligne 73 du fichier nfa.c.

4.15.1.12 nfa_elimeps()

```
nfa * nfa_elimeps (
          nfa * A)
```

Élimination des transitions epsilon d'un NFA.

Remarques

Si le NFA ne possède pas de transitions epsilon, alors une copie du NFA est retournée.

Renvoie

Un nouveau NFA sans transitions epsilon.

Définition à la ligne 78 du fichier nfa.c.

4.15.1.13 nfa_compute_runs()

```
barray * nfa_compute_runs (
          nfa * A,
          const char * w)
```

Calcule l'ensemble des états qui sont atteints par un mot dans un NFA.

Le retour se fait sous la forme d'un tableau de Booléens de taille nb_states où la case i est à true si et seulement si l'état i est atteint par le mot w.

Remarques

Si le mot contient des lettres qui ne sont pas dans l'alphabet du NFA, la fonction devra retourner un pointeur NULL.

Renvoie

La liste des états atteints avec le mot.

Définition à la ligne 83 du fichier nfa.c.

4.16 nfa.c

Aller à la documentation de ce fichier.

```
00001 #include "nfa.h"
00002 #include "printing.h"
00003
00004 void nfa_reset_state_names(nfa *A)
00005 {
00006     if (!A || !A->state_names)
00007     {
00008         return;
00009     }
00010
00011     for (int i = 0; i < A->transitions->nb_states; i++)
00012     {
00013         // printf("%d %s\n", A->transitions->nb_states, A->state_names[i]);
00014         free(A->state_names[i]);
00015     }
00016     free(A->state_names);
00017     A->state_names = NULL;
00018 }
00019
00020 /*********************************/
00021 /* Initialisation et suppression */
00022 /*********************************/
00023
00024 nfa *create_emptylang(void)
00025 {
00026     return NULL;
00027 }
00028
00029 nfa *create_sing_epsilon(void)
00030 {
00031     return NULL;
00032 }
00033
00034 nfa *create_sing_letter(char)
00035 {
00036     return NULL;
00037 }
00038
00039 void nfa_delete(nfa *)
00040 {
00041     return;
00042 }
00043
00044 /************************************/
00045 /* Opérations simples sur les NFAs */
00046 /************************************/
00047
00048 nfa *nfa_copy(nfa *)
00049 {
00050     return NULL;
00051 }
00052
00053 nfa *nfa_union(nfa *, nfa *)
00054 {
```

```
00055     return NULL;
00056 }
00057
00058 nfa *nfa_concat(nfa *, nfa *)
00059 {
00060     return NULL;
00061 }
00062
00063 nfa *nfa_star(nfa *)
00064 {
00065     return NULL;
00066 }
00067
00068 nfa *nfa_trim(nfa *)
00069 {
00070     return NULL;
00071 }
00072
00073 nfa *nfa_mirror(nfa *)
00074 {
00075     return NULL;
00076 }
00077
00078 nfa *nfa_elimeps(nfa *)
00079 {
00080     return NULL;
00081 }
00082
00083 barray *nfa_compute_runs(nfa *, const char *)
00084 {
00085     return NULL;
00086 }
```

4.17 Référence du fichier nfa.h

Implémentation des automates.

```
#include "graphs.h"
#include "type_basic.h"
#include "type_boolarray.h"
#include <stdarg.h>
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
```

Structures de données

— struct nfa

Type utilisé pour représenter un NFA.

Fonctions

— void nfa_reset_state_names (nfa ∗)

Efface les noms des états d'un NFA (si il y en a).

— nfa ∗ create_emptylang (void)

Crée un NFA qui reconnaît le langage vide.

— nfa ∗ create_sing_epsilon (void)

Créé un NFA qui reconnaît le singleton {epsilon}.

— nfa ∗ create_sing_letter (char letter)

Créé un NFA qui reconnaît un singleton contenant un mot d'une seule lettre.

— void nfa_delete (nfa ∗A)

Suppression d'un NFA.

— nfa ∗ nfa_copy (nfa ∗A)

Création d'une copie d'un NFA.

— nfa ∗ nfa_union (nfa ∗A1, nfa ∗A2)

Union de deux NFAs.

— nfa ∗ nfa_concat (nfa ∗A1, nfa ∗A2)

Concaténation de deux NFAs.

— nfa ∗ nfa_star (nfa ∗A)

Étoile de Kleene d'un NFA.

— nfa ∗ nfa_trim (nfa ∗A)

Élimination des états inutiles.

— nfa ∗ nfa_mirror (nfa ∗A)

Miroir d'un NFA.

— nfa ∗ nfa_elimeps (nfa ∗A)

Élimination des transitions epsilon d'un NFA.

— barray ∗ nfa_compute_runs (nfa ∗A, const char ∗w)

Calcule l'ensemble des états qui sont atteints par un mot dans un NFA.

4.17.1 Description détaillée

Implémentation des automates.

Contient le type utilisé pour représenter un automate ainsi que les fonctions basiques pour les manipuler.

Définition dans le fichier nfa.h.

4.17.2 Documentation des fonctions

4.17.2.1 nfa_reset_state_names()

```
void nfa_reset_state_names (
          nfa * A)
```

Efface les noms des états d'un NFA (si il y en a).

Définition à la ligne 4 du fichier nfa.c.

4.17.2.2 create_emptylang()

```
nfa * create_emptylang (
          void )
```

Crée un NFA qui reconnaît le langage vide.

Remarques

L'alphabet du NFA est vide également.

Renvoie

Le NFA.

Définition à la ligne 24 du fichier nfa.c.

4.17.2.3 create_sing_epsilon()

```
nfa * create_sing_epsilon (
          void )
```

Créé un NFA qui reconnaît le singleton {epsilon}.

Remarques

L'alphabet du NFA est vide.

Renvoie

Le NFA.

Définition à la ligne 29 du fichier nfa.c.

4.17.2.4 create_sing_letter()

```
nfa * create_sing_letter (
          char letter)
```

Créé un NFA qui reconnaît un singleton contenant un mot d'une seule lettre.

Remarques

L'alphabet du NFA contient une seule lettre.

Renvoie

Le NFA.

Définition à la ligne 34 du fichier nfa.c.

4.17.2.5 nfa_delete()

```
void nfa_delete (
          nfa * A)
```

Suppression d'un NFA.

Définition à la ligne 39 du fichier nfa.c.

4.17.2.6 nfa_copy()

```
nfa * nfa_copy (
            nfa * A)
```

Création d'une copie d'un NFA.

Renvoie

La copie.

Définition à la ligne 48 du fichier nfa.c.

4.17.2.7 nfa_union()

```
nfa * nfa_union (
            nfa * A1,
            nfa * A2)
```

Union de deux NFAs.

Attention

Les alphabets des deux NFAs peuvent être distincts. Dans ce cas, il faudra les étendre à l'union des deux alphabets.

Renvoie

Un nouveau NFA qui réalise l'union.

Définition à la ligne 53 du fichier nfa.c.

4.17.2.8 nfa_concat()

```
nfa * nfa_concat (
            nfa * A1,
            nfa * A2)
```

Concaténation de deux NFAs.

Attention

Les alphabets des deux NFAs peuvent être distincts. Dans ce cas, il faudra les étendre à l'union des deux alphabets.

Renvoie

Un nouveau NFA qui réalise la concaténation.

Définition à la ligne 58 du fichier nfa.c.

4.17.2.9 nfa_star()

```
nfa * nfa_star (
            nfa * A)
```

Étoile de Kleene d'un NFA.

Renvoie

Un nouveau NFA qui réalise l'étoile.

Définition à la ligne 63 du fichier nfa.c.

4.17.2.10 nfa_trim()

```
nfa * nfa_trim (
            nfa * A)
```

Élimination des états inutiles.

Remarques

Est inutile tout état inaccessible depuis un état initial ou depuis lequel on ne peut pas atteindre un état final.

Renvoie

Un nouveau NFA dans lequel on a éliminé les états inutiles.

Définition à la ligne 68 du fichier nfa.c.

4.17.2.11 nfa_mirror()

```
nfa * nfa_mirror (
            nfa * A)
```

Miroir d'un NFA.

Renvoie

Un nouveau NFA qui réalise le miroir.

Définition à la ligne 73 du fichier nfa.c.

4.17.2.12 nfa_elimeps()

```
nfa * nfa_elimeps (
          nfa * A)
```

Élimination des transitions epsilon d'un NFA.

Remarques

Si le NFA ne possède pas de transitions epsilon, alors une copie du NFA est retournée.

Renvoie

Un nouveau NFA sans transitions epsilon.

Définition à la ligne 78 du fichier nfa.c.

4.17.2.13 nfa_compute_runs()

```
barray * nfa_compute_runs (
          nfa * A,
          const char * w)
```

Calcule l'ensemble des états qui sont atteints par un mot dans un NFA.

Le retour se fait sous la forme d'un tableau de Booléens de taille nb_states où la case i est à true si et seulement si l'état i est atteint par le mot w.

Remarques

Si le mot contient des lettres qui ne sont pas dans l'alphabet du NFA, la fonction devra retourner un pointeur NULL.

Renvoie

La liste des états atteints avec le mot.

Définition à la ligne 83 du fichier nfa.c.

4.18 nfa.h

Aller à la documentation de ce fichier.

```
00001
00010
00011 #ifndef NFA_H
00012 #define NFA_H
00013
00014 #include "graphs.h"
00015 #include "type_basic.h"
00016 #include "type_boolarray.h"
00017 #include <stdarg.h>
00018 #include <stdbool.h>
00019 #include <stdio.h>
00020 #include <stdlib.h>
00021
00041 typedef struct
00042 {
00043     int nb_states;
00044     int nb_letters;
00045     bool eps_trans;
00046     ngraph *transitions;
00047     int *initials;
00048     int nb_initials;
00049     int *finals;
00050     int nb_finals;
00051     char *letter_names;
00052
00053     /* Optionnel*/
00054     char **state_names;
00055 } nfa;
00056
00057 /********************/
00058 /* Gestion des noms */
00059 /********************/
00060
00065 void nfa_reset_state_names(nfa * // Le NFA
00066 );
00067
00068 /*********************************/
00069 /* Initialisation et suppression */
00070 /*********************************/
00071
00082 nfa *create_emptylang(void);
00083
00094 nfa *create_sing_epsilon(void);
00095
00106 nfa *create_sing_letter(char letter // !< La lettre.
00107 );
00108
00113 void nfa_delete(nfa *A // !< Le NFA.
00114 );
00115
00116 /***********************************/
00117 /* Opérations simples sur les NFAs */
00118 /***********************************/
00119
```

```
00127 nfa *nfa_copy(nfa *A // !< Le NFA à copier.
00128 );
00129
00141 nfa *nfa_union(nfa *A1, // !< Le premier NFA.
00142                nfa *A2  // !< Le second NFA.
00143 );
00144
00156 nfa *nfa_concat(nfa *A1, // !< Le premier NFA.
00157                 nfa *A2  // !< Le second NFA.
00158 );
00159
00167 nfa *nfa_star(nfa *A // !< Le NFA.
00168 );
00169
00181 nfa *nfa_trim(nfa *A // !< Le NFA.
00182 );
00183
00191 nfa *nfa_mirror(nfa *A // !< Le NFA.
00192 );
00193
00205 nfa *nfa_elimeps(nfa *A // !< Le NFA.
00206 );
00207
00208 /************************/
00209 /* Information sur nfa */
00210 /************************/
00211
00227 barray *nfa_compute_runs(nfa *A,       // !< Le NFA.
00228                          const char *w // !< Le mot.
00229 );
00230
00231 #endif
```

4.19 Référence du fichier nfa_determi.c

```
#include "nfa_determi.h"
#include "printing.h"
#include "type_hash.h"
```

Fonctions

— nfa * nfa_determinize (nfa *, bool)

Déterminisation d'un automate.

— nfa * nfa_complement (nfa *)

Complémentation d'un automate.

4.19.1 Documentation des fonctions

4.19.1.1 nfa_determinize()

```
nfa * nfa_determinize (
          nfa * A,
          bool names)
```

Déterminisation d'un automate.

L'algorithme est basé sur le construction des sous-ensembles: chaque état de l'automate déterminisé correspond à un ensemble d'états de l'automate passé en entrée.

Remarques

Le Booléen passé en paramètre indique si on souhaite enregistrer les noms des états de l'automate déterminisé pour l'affichage. Le nom d'un état de l'automate déterminisé est l'ensemble d'états de l'automate original auquel il correspond. On devra donc construire ces noms lors de la déterminisation et les stocker dans le tableau des noms des états de l'automate déterminisé sous la forme d'une chaîne de caractères.

Attention

On ne générera QUE les états accessibles depuis l'état initial de l'automate déterminisé qu'on construit.

Renvoie

L'automate déterminisé.

Paramètres

A	Le NFA à déterminiser.
names	Un Booléen indiquant si on souhaite enregistrer les noms des états.

Définition à la ligne 9 du fichier nfa_determi.c.

4.19.1.2 nfa_complement()

```
nfa * nfa_complement (
          nfa * A)
```

Complémentation d'un automate.

Renvoie

L'automate complémenté.

Paramètres

A	Le NFA à complémenter.

Définition à la ligne 14 du fichier nfa_determi.c.

4.20 nfa_determi.c

Aller à la documentation de ce fichier.

```
00001 /*******************************************/
00002 /* Determinisation et Minimisation des NFAs */
00003 /*******************************************/
00004
00005 #include "nfa_determi.h"
00006 #include "printing.h"
00007 #include "type_hash.h"
00008
00009 nfa *nfa_determinize(nfa *, bool)
00010 {
00011     return NULL;
00012 }
00013
00014 nfa *nfa_complement(nfa *)
00015 {
00016     return NULL;
00017 }
```

4.21 Référence du fichier nfa_determi.h

Déterminisation et complémentation.

```
#include "nfa.h"
#include "printing.h"
```

Fonctions

— nfa * nfa_determinize (nfa *A, bool names)

Déterminisation d'un automate.

— nfa * nfa_complement (nfa *A)

Complémentation d'un automate.

4.21.1 Description détaillée

Déterminisation et complémentation.

Contient l'algorithme de déterminisation d'un automate basé sur la construction des sous-ensembles d'états et l'algorithme de complémentation.

Définition dans le fichier nfa_determi.h.

4.21.2 Documentation des fonctions

4.21.2.1 nfa_determinize()

```
nfa * nfa_determinize (
          nfa * A,
          bool names)
```

Déterminisation d'un automate.

L'algorithme est basé sur le construction des sous-ensembles: chaque état de l'automate déterminisé correspond à un ensemble d'états de l'automate passé en entrée.

Remarques

Le Booléen passé en paramètre indique si on souhaite enregistrer les noms des états de l'automate déterminisé pour l'affichage. Le nom d'un état de l'automate déterminisé est l'ensemble d'états de l'automate original auquel il correspond. On devra donc construire ces noms lors de la déterminisation et les stocker dans le tableau des noms des états de l'automate déterminisé sous la forme d'une chaîne de caractères.

Attention

On ne générera QUE les états accessibles depuis l'état initial de l'automate déterminisé qu'on construit.

Renvoie

L'automate déterminisé.

Paramètres

A	Le NFA à déterminiser.
names	Un Booléen indiquant si on souhaite enregistrer les noms des états.

Définition à la ligne 9 du fichier nfa_determi.c.

4.21.2.2 nfa_complement()

```
nfa * nfa_complement (
            nfa * A)
```

Complémentation d'un automate.

Renvoie

L'automate complémenté.

Paramètres

A	Le NFA à complémenter.

Définition à la ligne 14 du fichier nfa_determi.c.

4.22 nfa_determi.h

Aller à la documentation de ce fichier.

```
00001
00010
00011 #ifndef NFA_DETERMI_H
00012 #define NFA_DETERMI_H
00013
00014 #include "nfa.h"
00015 #include "printing.h"
00016
00038 nfa *nfa_determinize(nfa *A,
00039                      bool names
00040 );
00041
00049 nfa *nfa_complement(nfa *A
00050 );
00051
00052 #endif
```

4.23 Référence du fichier nfa_intersec.c

```
#include "nfa_intersec.h"
#include "printing.h"
#include "type_hash.h"
```

Fonctions

— nfa * nfa_intersect (nfa *, nfa *, bool)

Intersection de deux automates.

4.23.1 Documentation des fonctions

4.23.1.1 nfa_intersect()

```
nfa * nfa_intersect (
          nfa * A1,
          nfa * A2,
          bool names)
```

Intersection de deux automates.

L'algorithme est basé sur l'automate produit: chaque état de l'automate qu'on construit correspond à une paire `(q1,q2)` où `q1` est un état du premier automate passé en entrée et `q2` est un état du second automate passé en entrée.

Remarques

Le Booléen passé en paramètre indique si on souhaite enregistrer les noms des états de l'automate déterminisé pour l'affichage. Le nom d'un état de l'automate déterminisé est la paire d'états des automates passés en entrée auquel il correspond. On devra donc construire ces noms lors de l'intersection et les stocker dans le tableau des noms des états de l'automate produit sous la forme d'une chaîne de caractères.

Attention

On ne générera QUE les états accessibles depuis un état initial de l'automate produit qu'on construit.

Renvoie

L'automate qui réalise l'intersection.

Paramètres

A1	Le premier NFA à intersecter.
A2	Le second NFA à intersecter.
names	Un Booléen indiquant si on souhaite enregistrer les noms des états.

Définition à la ligne 9 du fichier nfa_intersec.c.

4.24 nfa_intersec.c

Aller à la documentation de ce fichier.

```
00001 /************************/
00002 /* Intersection of NFAs */
00003 /************************/
00004
00005 #include "nfa_intersec.h"
00006 #include "printing.h"
00007 #include "type_hash.h"
00008
00009 nfa *nfa_intersect(nfa *, nfa *, bool)
00010 {
00011     return NULL;
00012 }
```

4.25 Référence du fichier nfa_intersec.h

Intersection de NFAs.

```
#include "nfa.h"
```

Fonctions

— nfa * nfa_intersect (nfa *A1, nfa *A2, bool names)

Intersection de deux automates.

4.25.1 Description détaillée

Intersection de NFAs.

Contient l'algorithme qui permet de calculer l'intersection de deux NFAs. Il est basé sur l'automate produit.

Définition dans le fichier nfa_intersec.h.

4.25.2 Documentation des fonctions

4.25.2.1 nfa_intersect()

```
nfa * nfa_intersect (
          nfa * A1,
          nfa * A2,
          bool names)
```

Intersection de deux automates.

L'algorithme est basé sur l'automate produit: chaque état de l'automate qu'on construit correspond à une paire `(q1,q2)` où `q1` est un état du premier automate passé en entrée et `q2` est un état du second automate passé en entrée.

Remarques

Le Booléen passé en paramètre indique si on souhaite enregistrer les noms des états de l'automate déterminisé pour l'affichage. Le nom d'un état de l'automate déterminisé est la paire d'états des automates passés en entrée auquel il correspond. On devra donc construire ces noms lors de l'intersection et les stocker dans le tableau des noms des états de l'automate produit sous la forme d'une chaîne de caractères.

Attention

On ne générera QUE les états accessibles depuis un état initial de l'automate produit qu'on construit.

Renvoie

L'automate qui réalise l'intersection.

Paramètres

A1	Le premier NFA à intersecter.
A2	Le second NFA à intersecter.
names	Un Booléen indiquant si on souhaite enregistrer les noms des états.

Définition à la ligne 9 du fichier nfa_intersec.c.

4.26 nfa_intersec.h

Aller à la documentation de ce fichier.

```
00001
00010
00011 #ifndef NFA_INTERSEC_H
00012 #define NFA_INTERSEC_H
00013
00014 #include "nfa.h"
00015
00037 nfa *nfa_intersect(nfa *A1,
00038                    nfa *A2,
00039                    bool names
00040 );
00041
00042 #endif
```

4.27 Référence du fichier nfa_mccluskey.c

```
#include "nfa_mccluskey.h"
```

Fonctions

— regexp * nfa_mccluskey (nfa *)

Algorithme de Brzozowski-McCluskey.

4.27.1 Documentation des fonctions

4.27.1.1 nfa_mccluskey()

```
regexp * nfa_mccluskey (
            nfa * A)
```

Algorithme de Brzozowski-McCluskey.

Renvoie

Une expression régulière qui reconnaît le même langage que l'automate passé en entrée.

Paramètres

A	Un NFA quelconque.

Définition à la ligne 3 du fichier nfa_mccluskey.c.

4.28 nfa_mccluskey.c

Aller à la documentation de ce fichier.

```
00001 #include "nfa_mccluskey.h"
00002
00003 regexp *nfa_mccluskey(nfa *)
00004 {
00005     return NULL;
00006 }
```

4.29 Référence du fichier nfa_mccluskey.h

Calcul d'une expression régulière à partir d'un NFA avec l'algorithme de Brzozowski-McCluskey.

```
#include "nfa.h"
#include "regexp.h"
```

Fonctions

— regexp * nfa_mccluskey (nfa *A)

Algorithme de Brzozowski-McCluskey.

4.29.1 Description détaillée

Calcul d'une expression régulière à partir d'un NFA avec l'algorithme de Brzozowski-McCluskey.

Définition dans le fichier nfa_mccluskey.h.

4.29.2 Documentation des fonctions

4.29.2.1 nfa_mccluskey()

```
regexp * nfa_mccluskey (
            nfa * A)
```

Algorithme de Brzozowski-McCluskey.

Renvoie

Une expression régulière qui reconnaît le même langage que l'automate passé en entrée.

Paramètres

A	Un NFA quelconque.

Définition à la ligne 3 du fichier nfa_mccluskey.c.

4.30 nfa_mccluskey.h

Aller à la documentation de ce fichier.

```
00001
00006
00007 #ifndef MCCLUSKEY_H
00008 #define MCCLUSKEY_H
00009
00010 #include "nfa.h"
00011 #include "regexp.h"
00012
00020 regexp *nfa_mccluskey(nfa *A
00021 );
00022
00023 #endif
```

4.31 Référence du fichier nfa_minimization.c

```
#include "nfa_minimization.h"
```

Fonctions

— nfa * nfa_brzozowski (nfa *)

Algorithme de minimisation de Brzozowski.

4.31.1 Documentation des fonctions

4.31.1.1 nfa_brzozowski()

```
nfa * nfa_brzozowski (
            nfa * A)
```

Algorithme de minimisation de Brzozowski.

Renvoie

L'automate minimal du langage reconnu par le NFA pris en entrée.

Paramètres

A	Un NFA.

Définition à la ligne 7 du fichier nfa_minimization.c.

4.32 nfa_minimization.c

Aller à la documentation de ce fichier.

```
00001 #include "nfa_minimization.h"
00002
00003 /*******************************/
00004 /*+ Algorithme de Brzozowski +*/
00005 /*******************************/
00006
00007 nfa *nfa_brzozowski(nfa *)
00008 {
00009     return NULL;
00010 }
```

4.33 Référence du fichier nfa_minimization.h

Minimisation.

```
#include "nfa_determi.h"
#include "printing.h"
```

Fonctions

— nfa * nfa_brzozowski (nfa *A)

Algorithme de minimisation de Brzozowski.

4.33.1 Description détaillée

Minimisation.

Contient les deux algorithmes (indépendents) de minimisation d'un automate: Brzozowski et Hopcroft.

Remarques

Les deux algorithmes prennent n'importe quel NFA en entrée.

Définition dans le fichier nfa_minimization.h.

4.33.2 Documentation des fonctions

4.33.2.1 nfa_brzozowski()

```
nfa * nfa_brzozowski (
            nfa * A)
```

Algorithme de minimisation de Brzozowski.

Renvoie

L'automate minimal du langage reconnu par le NFA pris en entrée.

Paramètres

A	Un NFA.

Définition à la ligne 7 du fichier nfa_minimization.c.

4.34 nfa_minimization.h

Aller à la documentation de ce fichier.

```
00001
00012
00013 #ifndef MINIMIZATION_H
00014 #define MINIMIZATION_H
00015
00016 #include "nfa_determi.h"
00017 #include "printing.h"
00018
00026 nfa *nfa_brzozowski(nfa *A
00027 );
00028
00029 #endif
```

4.35 Référence du fichier parameters.h

Macros

— #define ADVANCED 0

Cette macro influe sur l'affichage des automates.

4.35.1 Documentation des macros

4.35.1.1 ADVANCED

```
#define ADVANCED 0
```

Cette macro influe sur l'affichage des automates.

- Si elle est définie à 0, les transitions dont les extrémités sont les mêmes sont affichées comme plusieurs flèches (autant de flèches que de transitions).
- Sinon, les transitions de mêmes extrémités sont regroupées en une seule flèche.

Pour utiliser la deuxième option, il est nécessaire d'avoir écrit la fonction ngraph_to_multi_edges() du fichier graphs.c.

Définition à la ligne 18 du fichier parameters.h.

4.36 parameters.h

Aller à la documentation de ce fichier.

```
00001 #ifndef PARAMETERS_H_
00002 #define PARAMETERS_H_
00003
00018 #define ADVANCED 0
00019
00020 #endif // PARAMETERS_H_
```

4.37 Référence du fichier regexp.c

```
#include "regexp.h"
```

Fonctions

— bool reg_issimple (regexp *)

Teste si une expression régulière est simple.

— void reg_free (regexp *)

Libération d'une expression régulière.

— regexp * reg_copy (regexp *)

Copie d'une expression régulière.

— regexp * reg_empty (void)

Création d'une expression régulière réduite au symbole définissant le langage vide.

— regexp * reg_epsilon (void)

Création d'une expression régulière réduite au symbole définissant le singleton contenant le mot-vide.

— regexp * reg_letter (char)

Création d'une expression régulière réduite à une seule lettre.

— regexp * reg_union (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur d'union.

— regexp * reg_inter (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur d'intersection.

— regexp * reg_concat (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur de concaténation.

— regexp * reg_star (regexp *)

Application de l'étoile de Kleene à une expression.

— regexp * reg_complement (regexp *)

Application de l'opérateur complément à une expression.

— static void reg_print_helper (regexp *r, regelem parent)

— void reg_print (regexp *r)

Affichage d'une expression régulière sur la sortie standard.

4.37.1 Documentation des fonctions

4.37.1.1 reg_issimple()

```
bool reg_issimple (
          regexp * )
```

Teste si une expression régulière est simple.

Remarques

Une expression régulière est simple si et seulement si elle ne contient pas les opérateurs "intersection" et "complément".

Renvoie

Un Booléen indiquant si l'expression passée en entrée est simple.

Définition à la ligne 3 du fichier regexp.c.

4.37.1.2 reg_free()

```
void reg_free (
           regexp * )
```

Libération d'une expression régulière.

Définition à la ligne 8 du fichier regexp.c.

4.37.1.3 reg_copy()

```
regexp * reg_copy (
           regexp * )
```

Copie d'une expression régulière.

Renvoie

Une copie de l'expression régulière passée en entrée.

Définition à la ligne 13 du fichier regexp.c.

4.37.1.4 reg_empty()

```
regexp * reg_empty (
           void )
```

Création d'une expression régulière réduite au symbole définissant le langage vide.

Renvoie

L'expression régulière.

Définition à la ligne 18 du fichier regexp.c.

4.37.1.5 reg_epsilon()

```
regexp * reg_epsilon (
           void )
```

Création d'une expression régulière réduite au symbole définissant le singleton contenant le mot-vide.

Renvoie

L'expression régulière.

Définition à la ligne 23 du fichier regexp.c.

4.37.1.6 reg_letter()

```
regexp * reg_letter (
           char )
```

Création d'une expression régulière réduite à une seule lettre.

Renvoie

L'expression régulière.

Définition à la ligne 28 du fichier regexp.c.

4.37.1.7 reg_union()

```
regexp * reg_union (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur d'union.

Renvoie

L'expression régulière résultante.

Définition à la ligne 33 du fichier regexp.c.

4.37.1.8 reg_inter()

```
regexp * reg_inter (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur d'intersection.

Renvoie

L'expression régulière résultante.

Définition à la ligne 38 du fichier regexp.c.

4.37.1.9 reg_concat()

```
regexp * reg_concat (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur de concaténation.

Renvoie

L'expression régulière résultante.

Définition à la ligne 43 du fichier regexp.c.

4.37.1.10 reg_star()

```
regexp * reg_star (
           regexp * )
```

Application de l'étoile de Kleene à une expression.

Renvoie

L'expression régulière résultante.

Définition à la ligne 48 du fichier regexp.c.

4.37.1.11 reg_complement()

```
regexp * reg_complement (
           regexp * )
```

Application de l'opérateur complément à une expression.

Renvoie

L'expression régulière résultante.

Définition à la ligne 53 du fichier regexp.c.

4.37.1.12 reg_print_helper()

```
void reg_print_helper (
           regexp * r,
           regelem parent) [static]
```

Définition à la ligne 58 du fichier regexp.c.

4.37.1.13 reg_print()

```
void reg_print (
           regexp * r)
```

Affichage d'une expression régulière sur la sortie standard.

Attention

Cette fonction est déjà écrite pour vous. Vous ne devez PAS la modifier.

Paramètres

r	L'expression à afficher.

Définition à la ligne 167 du fichier regexp.c.

4.38 regexp.c

Aller à la documentation de ce fichier.

```
00001 #include "regexp.h"
00002
00003 bool reg_issimple(regexp *)
00004 {
00005     return false;
00006 }
00007
00008 void reg_free(regexp *)
00009 {
00010     return;
00011 }
00012
00013 regexp *reg_copy(regexp *)
00014 {
00015     return NULL;
00016 }
00017
00018 regexp *reg_empty(void)
00019 {
00020     return NULL;
00021 }
00022
00023 regexp *reg_epsilon(void)
00024 {
00025     return NULL;
00026 }
00027
00028 regexp *reg_letter(char)
00029 {
00030     return NULL;
00031 }
00032
00033 regexp *reg_union(regexp *, regexp *)
00034 {
00035     return NULL;
00036 }
00037
00038 regexp *reg_inter(regexp *, regexp *)
00039 {
00040     return NULL;
00041 }
00042
00043 regexp *reg_concat(regexp *, regexp *)
00044 {
00045     return NULL;
00046 }
00047
00048 regexp *reg_star(regexp *)
00049 {
00050     return NULL;
00051 }
00052
00053 regexp *reg_complement(regexp *)
00054 {
```

```
00055     return NULL;
00056 }
00057
00058 static void reg_print_helper(regexp *r, regelem parent)
00059 {
00060     TRACE("reg_print_helper");
00061
00062     if (r == NULL)
00063     {
00064         return;
00065     }
00066
00067     switch (r->op)
00068     {
00069     case CHAR:
00070         if (r->letter == ' ')
00071         {
00072             fprintf(stdout, "%s", "␣");
00073         }
00074         else
00075         {
00076             fprintf(stdout, "%c", r->letter);
00077         }
00078         break;
00079
00080     case EMPTY:
00081         print_color("∅", BLUE, stdout);
00082         break;
00083
00084     case EPSILON:
00085         print_color("ε", YELLOW, stdout);
00086         break;
00087
00088     case UNION:
00089         DEBUG("Union");
00090
00091         if (parent != UNION && parent != NONE)
00092         {
00093             print_color("(", CYAN, stdout);
00094         }
00095         reg_print_helper(r->left, UNION);
00096         print_color(" + ", RED, stdout);
00097         reg_print_helper(r->right, UNION);
00098         if (parent != UNION && parent != NONE)
00099         {
00100             print_color(")", CYAN, stdout);
00101         }
00102         break;
00103
00104     case INTER:
00105         if (parent != INTER && parent != NONE)
00106         {
00107             print_color("(", CYAN, stdout);
00108         }
00109         reg_print_helper(r->left, INTER);
00110         printf(" ∩ ");
00111         reg_print_helper(r->right, INTER);
00112         if (parent != INTER && parent != NONE)
```

```
00113         {
00114             print_color(")", CYAN, stdout);
00115         }
00116         break;
00117
00118     case CONCAT:
00119         DEBUG("Concat");
00120
00121         if (parent != CONCAT && parent != UNION && parent != NONE)
00122         {
00123             print_color("(", CYAN, stdout);
00124         }
00125         reg_print_helper(r->left, CONCAT);
00126         reg_print_helper(r->right, CONCAT);
00127         if (parent != CONCAT && parent != UNION && parent != NONE)
00128         {
00129             print_color(")", CYAN, stdout);
00130         }
00131         break;
00132
00133     case STAR:
00134         DEBUG("Star");
00135
00136         if (r->left->op == CHAR || r->left->op == EPSILON || r->left->op ==
      EMPTY)
00137         {
00138             reg_print_helper(r->left, STAR);
00139             print_color("*", CYAN, stdout);
00140         }
00141         else
00142         {
00143             reg_print_helper(r->left, STAR);
00144             print_color("*", CYAN, stdout);
00145         }
00146         break;
00147
00148     case COMPLEMENT:
00149         if (parent != NONE)
00150         {
00151             print_color("(", CYAN, stdout);
00152         }
00153         print_color("ň", RED, stdout);
00154         reg_print_helper(r->left, COMPLEMENT);
00155         if (parent != NONE)
00156         {
00157             print_color(")", CYAN, stdout);
00158         }
00159         break;
00160
00161     default:
00162         CRITICAL("Unknown regexp operator (%d)", r->op);
00163         break;
00164     }
00165 }
00166
00167 void reg_print(regexp *r)
00168 {
00169     TRACE("reg_print");
```

```
00170
00171     reg_print_helper(r, NONE);
00172     printf("\n");
00173 }
```

4.39 Référence du fichier regexp.h

Implémentation des expressions régulières.

```
#include "alloc.h"
#include "error.h"
#include "tools.h"
#include "type_basic.h"
#include <assert.h>
#include <ctype.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>
```

Structures de données

— struct regexp

Type utilisé pour représenter un noeud d'une expression régulière.

Définitions de type

— typedef struct regexp regexp

Type utilisé pour représenter un noeud d'une expression régulière.

Énumérations

— enum regelem {
EMPTY , EPSILON , CHAR , UNION ,
INTER , COMPLEMENT , CONCAT , STAR ,
NONE }

Les différents opérateurs utilisables dans une expression régulière.

Fonctions

— bool reg_issimple (regexp *)

Teste si une expression régulière est simple.

— void reg_free (regexp *)

Libération d'une expression régulière.

— regexp * reg_copy (regexp *)

Copie d'une expression régulière.

— regexp * reg_empty (void)

Création d'une expression régulière réduite au symbole définissant le langage vide.

— regexp * reg_epsilon (void)

Création d'une expression régulière réduite au symbole définissant le singleton contenant le mot-vide.

— regexp * reg_letter (char)

Création d'une expression régulière réduite à une seule lettre.

— regexp * reg_union (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur d'union.

— regexp * reg_inter (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur d'intersection.

— regexp * reg_concat (regexp *, regexp *)

Combinaison de deux expressions avec l'opérateur de concaténation.

— regexp * reg_star (regexp *)

Application de l'étoile de Kleene à une expression.

— regexp * reg_complement (regexp *)

Application de l'opérateur complément à une expression.

— void reg_print (regexp *)

Affichage d'une expression régulière sur la sortie standard.

4.39.1 Description détaillée

Implémentation des expressions régulières.

Contient le type utilisé pour représenter une expression régulière ainsi que les fonctions basiques pour les manipuler.

Définition dans le fichier regexp.h.

4.39.2 Documentation des définitions de type

4.39.2.1 regexp

```
typedef struct regexp regexp
```

Type utilisé pour représenter un noeud d'une expression régulière.

Les champs left et right sont utilisés pour représenter les sous-expressions gauche et droite (si elles existent):

- Pour les opérateurs unaire (COMPLEMENT et STAR), seul le champ left est utilisé.
- Pour les opérateurs binaires (UNION, INTER et CONCAT), les deux champs sont utilisés.
- Pour les opérandes (EMPTY, EPSILON et CHAR), aucun des deux champs n'est utilisé. Quand un champ n'est pas utilisé, il doit avoir la valeur NULL.

4.39.3 Documentation du type de l'énumération

4.39.3.1 regelem

```
enum regelem
```

Les différents opérateurs utilisables dans une expression régulière.

Valeurs énumérées

EMPTY	L'ensemble vide.
EPSILON	Le mot vide.
CHAR	Une lettre.
UNION	L'union de deux expressions régulières.
INTER	L'intersection de deux expressions régulières.
COMPLEMENT	Le complément d'une expression régulière.

CONCAT	La concaténation de deux expressions régulières.
STAR	L'étoile de Kleene d'une expression régulière.
NONE	Utilisé uniquement pour simplifier l'affichage.

Définition à la ligne 27 du fichier regexp.h.

4.39.4 Documentation des fonctions

4.39.4.1 reg_issimple()

```
bool reg_issimple (
           regexp * )
```

Teste si une expression régulière est simple.

Remarques

Une expression régulière est simple si et seulement si elle ne contient pas les opérateurs "intersection" et "complément".

Renvoie

Un Booléen indiquant si l'expression passée en entrée est simple.

Définition à la ligne 3 du fichier regexp.c.

4.39.4.2 reg_free()

```
void reg_free (
           regexp * )
```

Libération d'une expression régulière.

Définition à la ligne 8 du fichier regexp.c.

4.39.4.3 reg_copy()

```
regexp * reg_copy (
           regexp * )
```

Copie d'une expression régulière.

Renvoie

Une copie de l'expression régulière passée en entrée.

Définition à la ligne 13 du fichier regexp.c.

4.39.4.4 reg_empty()

```
regexp * reg_empty (
           void )
```

Création d'une expression régulière réduite au symbole définissant le langage vide.

Renvoie

L'expression régulière.

Définition à la ligne 18 du fichier regexp.c.

4.39.4.5 reg_epsilon()

```
regexp * reg_epsilon (
           void )
```

Création d'une expression régulière réduite au symbole définissant le singleton contenant le mot-vide.

Renvoie

L'expression régulière.

Définition à la ligne 23 du fichier regexp.c.

4.39.4.6 reg_letter()

```
regexp * reg_letter (
           char )
```

Création d'une expression régulière réduite à une seule lettre.

Renvoie

L'expression régulière.

Définition à la ligne 28 du fichier regexp.c.

4.39.4.7 reg_union()

```
regexp * reg_union (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur d'union.

Renvoie

L'expression régulière résultante.

Définition à la ligne 33 du fichier regexp.c.

4.39.4.8 reg_inter()

```
regexp * reg_inter (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur d'intersection.

Renvoie

L'expression régulière résultante.

Définition à la ligne 38 du fichier regexp.c.

4.39.4.9 reg_concat()

```
regexp * reg_concat (
           regexp * ,
           regexp * )
```

Combinaison de deux expressions avec l'opérateur de concaténation.

Renvoie

L'expression régulière résultante.

Définition à la ligne 43 du fichier regexp.c.

4.39.4.10 reg_star()

```
regexp * reg_star (
           regexp * )
```

Application de l'étoile de Kleene à une expression.

Renvoie

L'expression régulière résultante.

Définition à la ligne 48 du fichier regexp.c.

4.39.4.11 reg_complement()

```
regexp * reg_complement (
           regexp * )
```

Application de l'opérateur complément à une expression.

Renvoie

L'expression régulière résultante.

Définition à la ligne 53 du fichier regexp.c.

4.39.4.12 reg_print()

```
void reg_print (
           regexp * r)
```

Affichage d'une expression régulière sur la sortie standard.

Attention

Cette fonction est déjà écrite pour vous. Vous ne devez PAS la modifier.

Paramètres

r	L'expression à afficher.

Définition à la ligne 167 du fichier regexp.c.

4.40 regexp.h

Aller à la documentation de ce fichier.

```
00001
00010 #ifndef REGEXP_H_
00011 #define REGEXP_H_
00012
00013 #include "alloc.h"
00014 #include "error.h"
00015 #include "tools.h"
00016 #include "type_basic.h"
00017 #include <assert.h>
00018 #include <ctype.h>
00019 #include <stdbool.h>
00020 #include <stdio.h>
00021 #include <string.h>
```

```
00022
00027 typedef enum
00028 {
00029     EMPTY,
00030     EPSILON,
00031     CHAR,
00032     UNION,
00033     INTER,
00034     COMPLEMENT,
00035     CONCAT,
00036     STAR,
00037     NONE
00038 } regelem;
00039
00052 typedef struct regexp
00053 {
00054     regelem op;
00055     struct regexp *left;
00056     struct regexp *right;
00057     char letter;
00058 } regexp;
00059
00070 bool reg_issimple(regexp *
00071 );
00072
00077 void reg_free(regexp *
00078 );
00079
00087 regexp *reg_copy(regexp *
00088 );
00089
00097 regexp *reg_empty(void);
00098
00106 regexp *reg_epsilon(void);
00107
00115 regexp *reg_letter(char
00116 );
00117
00125 regexp *reg_union(regexp *,
00126                   regexp *
00127 );
00128
00136 regexp *reg_inter(regexp *,
00137                   regexp *
00138 );
00139
00147 regexp *reg_concat(regexp *,
00148                    regexp *
00149 );
00150
00158 regexp *reg_star(regexp *
00159 );
00160
00168 regexp *reg_complement(regexp *
00169 );
00170
00178 void reg_print(regexp *
00179 );
```

```
00180
00181 #endif // REGEXP_H_
```

4.41 Référence du fichier regexp_thompson.c

```
#include "regexp_thompson.h"
```

Fonctions

— nfa * reg_thompson (regexp *)

Algorithme de Thompson.

4.41.1 Documentation des fonctions

4.41.1.1 reg_thompson()

```
nfa * reg_thompson (
           regexp * )
```

Algorithme de Thompson.

Calcule un récursivement un NFA à partir d'une expression régulière.

Remarques

Contrairement à l'algorithme de Glushkov, l'algorithme de Thompson peut prendre une expression régulière généralisée (qui utilise l'intersection et le complément) en entrée.

Attention

Implémenter cet algorithme requiert d'avoir écrit au préalable la plupart des fonctions qui manipulent des NFAs (création d'un NFA basique, union, étoile de Kleene,...).

Renvoie

Un NFA qui reconnaît le même langage que l'expression régulière passée en entrée.

Définition à la ligne 3 du fichier regexp_thompson.c.

4.42 regexp_thompson.c

Aller à la documentation de ce fichier.

```
00001 #include "regexp_thompson.h"
00002
00003 nfa *reg_thompson(regexp *)
00004 {
00005     return NULL;
00006 }
```

4.43 Référence du fichier regexp_thompson.h

Construction d'un NFA à partir d'une expression régulière avec l'algorithme de Thompson.

```
#include "error.h"
#include "nfa.h"
#include "nfa_intersec.h"
#include "nfa_minimization.h"
#include "regexp.h"
#include "type_boolarray.h"
#include <stdlib.h>
```

Fonctions

— nfa * reg_thompson (regexp *)

Algorithme de Thompson.

4.43.1 Description détaillée

Construction d'un NFA à partir d'une expression régulière avec l'algorithme de Thompson.

Définition dans le fichier regexp_thompson.h.

4.43.2 Documentation des fonctions

4.43.2.1 reg_thompson()

```
nfa * reg_thompson (
            regexp * )
```

Algorithme de Thompson.

Calcule un récursivement un NFA à partir d'une expression régulière.

Remarques

Contrairement à l'algorithme de Glushkov, l'algorithme de Thompson peut prendre une expression régulière généralisée (qui utilise l'intersection et le complément) en entrée.

Attention

Implémenter cet algorithme requiert d'avoir écrit au préalable la plupart des fonctions qui manipulent des NFAs (création d'un NFA basique, union, étoile de Kleene,...).

Renvoie

Un NFA qui reconnaît le même langage que l'expression régulière passée en entrée.

Définition à la ligne 3 du fichier regexp_thompson.c.

4.44 regexp_thompson.h

Aller à la documentation de ce fichier.

```
00001
00006
00007 #ifndef REGEXP_THOMPSON_H_
00008 #define REGEXP_THOMPSON_H_
00009
00010 #include "error.h"
00011 #include "nfa.h"
00012 #include "nfa_intersec.h"
00013 #include "nfa_minimization.h"
00014 #include "regexp.h"
00015 #include "type_boolarray.h"
00016 #include <stdlib.h>
00017
00036 nfa *reg_thompson(regexp *
00037 );
00038
00039 #endif // REGEXP_THOMPSON_H_
```

4.45 Référence du fichier type_basic.h

Définitions des types basiques.

```
#include "string.h"
```

Macros

- #define max(a, b)
- #define min(a, b)

Définitions de type

- typedef unsigned char uchar
- typedef unsigned short ushort
- typedef unsigned int uint
- typedef unsigned long ulong

4.45.1 Description détaillée

Définitions des types basiques.

Définition dans le fichier type_basic.h.

4.45.2 Documentation des macros

4.45.2.1 max

```
#define max(
            a,
            b)
```

Valeur :

```
(((a) > (b)) ? (a) : (b))
```

Définition à la ligne 23 du fichier type_basic.h.

4.45.2.2 min

```
#define min(
            a,
            b)
```

Valeur :

```
(((a) < (b)) ? (a) : (b))
```

Définition à la ligne 27 du fichier type_basic.h.

4.45.3 Documentation des définitions de type

4.45.3.1 uchar

```
typedef unsigned char uchar
```

Définition à la ligne 18 du fichier type_basic.h.

4.45.3.2 ushort

```
typedef unsigned short ushort
```

Définition à la ligne 19 du fichier type_basic.h.

4.45.3.3 uint

```
typedef unsigned int uint
```

Définition à la ligne 20 du fichier type_basic.h.

4.45.3.4 ulong

```
typedef unsigned long ulong
```

Définition à la ligne 21 du fichier type_basic.h.

4.46 type_basic.h

Aller à la documentation de ce fichier.

```
00001
00006
00007 #ifndef TYPES_H
00008 #define TYPES_H
00009
00010  /*  _______   ______  _____ ____    ____    _    ____ ___ ___  _   _ _____ ____
      */
00011  /* |_   _\ \ / /  _ \| ____/ ___|  | __ )  / \  / ___|_ _/ _ \| | | | ____/ ___|
      */
00012  /*   | |  \ V /| |_) |  _| \___ \  |  _ \ / _ \ \___ \| | | | | | | |  _| \___ \
      */
00013  /*   | |   | | |  __/| |___ ___) | | |_) / ___ \ ___) | | |_| | |_| | |___ ___)
      | */
00014  /*   |_|   |_| |_|   |_____|____/  |____/_/   \_\____/___\__\_\\___/|_____|____/
      */
```

```
00015
00016 #include "string.h"
00017
00018 typedef unsigned char uchar;
00019 typedef unsigned short ushort;
00020 typedef unsigned int uint;
00021 typedef unsigned long ulong;
00022 #ifndef max
00023 #define max(a, b) (((a) > (b)) ? (a) : (b))
00024 #endif
00025
00026 #ifndef min
00027 #define min(a, b) (((a) < (b)) ? (a) : (b))
00028 #endif
00029
00030 #endif
```

4.47 Référence du fichier type_boolarray.c

`#include "type_boolarray.h"`

Fonctions

— barray * create_barray (int)

Création d'un tableau de Booléens.

— int getsize_barray (barray *)

Récupération de la taille d'un tableau de Booléens.

— void delete_barray (barray *)

Libération d'un tableau de Booléens.

— void settrue_barray (barray *, int)

Affectation de la valeur `true` dans une cellule d'un tableau de Booléens.

— void setfalse_barray (barray *, int)

Affectation de la valeur `false` dans une cellule d'un tableau de Booléens.

— bool getval_barray (barray *, int)

Affectation de la valeur "true" dans une cellule d'un tableau de Booléens.

— barray * or_barray (barray *, barray *)

Disjonction (OU bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

— barray * and_barray (barray *, barray *)

Conjonction (ET bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

— barray * copy_barray (barray *)

Copie d'un tableau de Booléens.

4.47.1 Documentation des fonctions

4.47.1.1 create_barray()

```
barray * create_barray (
           int )
```

Création d'un tableau de Booléens.

Attention

Toutes les cellules du tableau créé doivent être initialisées à la valeur `false`.

Renvoie

Le tableau créé.

Définition à la ligne 3 du fichier type_boolarray.c.

4.47.1.2 getsize_barray()

```
int getsize_barray (
          barray * )
```

Récupération de la taille d'un tableau de Booléens.

Remarques

On parle ici du nombre de Booléens stockés (pas de la taille réelle du tableau).

Renvoie

La taille du tableau de Booléens.

Définition à la ligne 8 du fichier type_boolarray.c.

4.47.1.3 delete_barray()

```
void delete_barray (
           barray * )
```

Libération d'un tableau de Booléens.

Définition à la ligne 13 du fichier type_boolarray.c.

4.47.1.4 settrue_barray()

```
void settrue_barray (
           barray * ,
           int )
```

Affectation de la valeur `true` dans une cellule d'un tableau de Booléens.

Définition à la ligne 18 du fichier type_boolarray.c.

4.47.1.5 setfalse_barray()

```
void setfalse_barray (
           barray * ,
           int )
```

Affectation de la valeur `false` dans une cellule d'un tableau de Booléens.

Définition à la ligne 23 du fichier type_boolarray.c.

4.47.1.6 getval_barray()

```
bool getval_barray (
           barray * ,
           int )
```

Affectation de la valeur "true" dans une cellule d'un tableau de Booléens.

Définition à la ligne 28 du fichier type_boolarray.c.

4.47.1.7 or_barray()

```
barray * or_barray (
           barray * ,
           barray * )
```

Disjonction (OU bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

Attention

Si les deux tableaux passés en entrée n'ont pas la même taille, on retournera NULL.

Renvoie

Un nouveau tableau correspondant à la disjonction cellule par cellule des deux tableaux passés en entrée.

Définition à la ligne 33 du fichier type_boolarray.c.

4.47.1.8 and_barray()

```
barray * and_barray (
           barray * ,
           barray * )
```

Conjonction (ET bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

Attention

Si les deux tableaux passés en entrée n'ont pas la même taille, on retournera NULL.

Renvoie

Un nouveau tableau correspondant à la conjonction cellule par cellule des deux tableaux passés en entrée.

Définition à la ligne 37 du fichier type_boolarray.c.

4.47.1.9 copy_barray()

```
barray * copy_barray (
           barray * )
```

Copie d'un tableau de Booléens.

Renvoie

La copie.

Définition à la ligne 42 du fichier type_boolarray.c.

4.48 type_boolarray.c

Aller à la documentation de ce fichier.

```
00001 #include "type_boolarray.h"
00002
00003 barray *create_barray(int)
00004 {
00005     return NULL;
00006 }
00007
00008 int getsize_barray(barray *)
00009 {
00010     return 0;
00011 }
00012
00013 void delete_barray(barray *)
00014 {
00015     return;
00016 }
00017
00018 void settrue_barray(barray *, int)
00019 {
00020     return;
00021 }
00022
00023 void setfalse_barray(barray *, int)
00024 {
00025     return;
00026 }
00027
00028 bool getval_barray(barray *, int)
00029 {
00030     return false;
00031 }
00032
00033 barray *or_barray(barray *, barray *)
00034 {
00035     return NULL;
00036 }
00037 barray *and_barray(barray *, barray *)
00038 {
00039     return NULL;
00040 }
00041
00042 barray *copy_barray(barray *)
00043 {
00044     return NULL;
00045 }
```

4.49 Référence du fichier type_boolarray.h

Implémentation des tableaux de Booléens.

```
#include "tools.h"
```

Structures de données

— struct barray

Type utilisé pour représenter un tableau de Booléens.

Fonctions

— barray * create_barray (int)

Création d'un tableau de Booléens.

— int getsize_barray (barray *)

Récupération de la taille d'un tableau de Booléens.

— void delete_barray (barray *)

Libération d'un tableau de Booléens.

— void settrue_barray (barray *, int)

Affectation de la valeur `true` dans une cellule d'un tableau de Booléens.

— void setfalse_barray (barray *, int)

Affectation de la valeur `false` dans une cellule d'un tableau de Booléens.

— bool getval_barray (barray *, int)

Affectation de la valeur "true" dans une cellule d'un tableau de Booléens.

— barray * or_barray (barray *, barray *)

Disjonction (OU bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

— barray * and_barray (barray *, barray *)

Conjonction (ET bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

— barray * copy_barray (barray *)

Copie d'un tableau de Booléens.

4.49.1 Description détaillée

Implémentation des tableaux de Booléens.

Définition dans le fichier type_boolarray.h.

4.49.2 Documentation des fonctions

4.49.2.1 create_barray()

```
barray * create_barray (
           int )
```

Création d'un tableau de Booléens.

Attention

Toutes les cellules du tableau créé doivent être initialisées à la valeur `false`.

Renvoie

Le tableau créé.

Définition à la ligne 3 du fichier type_boolarray.c.

4.49.2.2 getsize_barray()

```
int getsize_barray (
           barray * )
```

Récupération de la taille d'un tableau de Booléens.

Remarques

On parle ici du nombre de Booléens stockés (pas de la taille réelle du tableau).

Renvoie

La taille du tableau de Booléens.

Définition à la ligne 8 du fichier type_boolarray.c.

4.49.2.3 delete_barray()

```
void delete_barray (
           barray * )
```

Libération d'un tableau de Booléens.

Définition à la ligne 13 du fichier type_boolarray.c.

4.49.2.4 settrue_barray()

```
void settrue_barray (
           barray * ,
           int )
```

Affectation de la valeur `true` dans une cellule d'un tableau de Booléens.

Définition à la ligne 18 du fichier type_boolarray.c.

4.49.2.5 setfalse_barray()

```
void setfalse_barray (
           barray * ,
           int )
```

Affectation de la valeur `false` dans une cellule d'un tableau de Booléens.

Définition à la ligne 23 du fichier type_boolarray.c.

4.49.2.6 getval_barray()

```
bool getval_barray (
           barray * ,
           int )
```

Affectation de la valeur "true" dans une cellule d'un tableau de Booléens.

Définition à la ligne 28 du fichier type_boolarray.c.

4.49.2.7 or_barray()

```
barray * or_barray (
           barray * ,
           barray * )
```

Disjonction (OU bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

Attention

Si les deux tableaux passés en entrée n'ont pas la même taille, on retournera NULL.

Renvoie

Un nouveau tableau correspondant à la disjonction cellule par cellule des deux tableaux passés en entrée.

Définition à la ligne 33 du fichier type_boolarray.c.

4.49.2.8 and_barray()

```
barray * and_barray (
           barray * ,
           barray * )
```

Conjonction (ET bit à bit) cellule par cellule de deux tableaux de Booléens de même taille.

Attention

Si les deux tableaux passés en entrée n'ont pas la même taille, on retournera NULL.

Renvoie

Un nouveau tableau correspondant à la conjonction cellule par cellule des deux tableaux passés en entrée.

Définition à la ligne 37 du fichier type_boolarray.c.

4.49.2.9 copy_barray()

```
barray * copy_barray (
           barray * )
```

Copie d'un tableau de Booléens.

Renvoie

La copie.

Définition à la ligne 42 du fichier type_boolarray.c.

4.50 type_boolarray.h

Aller à la documentation de ce fichier.

```
00001
00006 #ifndef BOOLARRAY_H_
00007 #define BOOLARRAY_H_
00008
00009 #include "tools.h"
00010
00022
00023 typedef struct
00024 {
00025     uchar *array;
00026     int size;
00027     int size_array;
```

```
00028 } barray;
00029
00040 barray *create_barray(int
00041 );
00042
00053 int getsize_barray(barray *
00054 );
00055
00060 void delete_barray(barray *
00061 );
00062
00067 void settrue_barray(barray *,
00068                     int
00069 );
00070
00075 void setfalse_barray(barray *,
00076                      int
00077 );
00078
00083 bool getval_barray(barray *,
00084                    int
00085 );
00086
00097 barray *or_barray(barray *,
00098                   barray *
00099 );
00100
00111 barray *and_barray(barray *,
00112                    barray *
00113 );
00114
00122 barray *copy_barray(barray *
00123 );
00124
00125 #endif
```

4.51 Référence du fichier type_hash.c

```
#include "type_hash.h"
#include "alloc.h"
#include <limits.h>
```

Fonctions

— hash_table * create_hash_table (int element_size)

Crée une nouvelle table de hachage vide.

— void delete_hash_table (hash_table *ht)

Supprime une table de hachage.

— static int hash_function (int *elements, int size, int hash_capacity)

— static void hash_table_rehash (hash_table *ht, int *elements)

— static bool hash_array_compare (int *a, int *b, int size)

— bool hash_table_insert (hash_table *ht, int *elements, int i, int *ret)

Tente d'insérer un élément dans la table de hachage.

4.51.1 Documentation des fonctions

4.51.1.1 create_hash_table()

```
hash_table * create_hash_table (
          int element_size)
```

Crée une nouvelle table de hachage vide.

Renvoie

La table de hachage vide créée.

Paramètres

element_size	Nombre de valeurs utilisées pour encoder un seul élément dans le tableau de stockage externe.

Définition à la ligne 5 du fichier type_hash.c.

4.51.1.2 delete_hash_table()

```
void delete_hash_table (
          hash_table * ht)
```

Supprime une table de hachage.

Paramètres

ht	La table de hachage à supprimer.

Définition à la ligne 25 du fichier type_hash.c.

4.51.1.3 hash_function()

```
int hash_function (
            int * elements,
            int size,
            int hash_capacity) [static]
```

Paramètres

elements	The element to hash.
size	Size of the element (number of integers).
hash_capacity	Capacity of the hash table.

Définition à la ligne 36 du fichier type_hash.c.

4.51.1.4 hash_table_rehash()

```
void hash_table_rehash (
            hash_table * ht,
            int * elements) [static]
```

Définition à la ligne 49 du fichier type_hash.c.

4.51.1.5 hash_array_compare()

```
bool hash_array_compare (
            int * a,
            int * b,
            int size) [static]
```

Définition à la ligne 87 du fichier type_hash.c.

4.51.1.6 hash_table_insert()

```
bool hash_table_insert (
            hash_table * ht,
            int * elements,
            int i,
            int * ret)
```

Tente d'insérer un élément dans la table de hachage.

Si l'élément est déjà présent dans la table de hachage, l'insertion n'est pas effectuée. De plus l'indice du premier élément identique trouvé est stocké dans la variable pointée par ret (si ret n'est pas NULL).

Remarques

> L'indice i fait référence à la position de l'élément dans le tableau de stockage externe. Puisque chaque élément est encodé comme une séquence d'entiers de taille fixe (le champ element↩ _size), la table de hachage va en fait examiner la séquence de element_size valeurs entières commençant à la position i * element_size dans le tableau de stockage externe.

Renvoie

> true si l'élément a été inséré, false s'il était déjà présent. Dans ce dernier cas, la table de hachage n'est pas modifiée.

Paramètres

ht	La table de hachage.
elements	Tableau de stockage externe contenant tous les éléments.
i	Indice de l'élément à insérer dans le tableau de stockage externe.
ret	Pointeur vers un entier où sera stocké l'indice de l'élément existant (si déjà présent).

Définition à la ligne 96 du fichier type_hash.c.

4.52 type_hash.c

Aller à la documentation de ce fichier.

```
00001 #include "type_hash.h"
00002 #include "alloc.h"
00003 #include <limits.h>
00004
00005 hash_table *create_hash_table(int element_size) {
00006     // Allocating memory for the hash table structure.
00007     hash_table *ht;
00008     CREATE_MEMORY(ht, 1);
00009
00010     // Initializing the hash table.
00011     ht->hash_capacity = 128; // Initial capacity is set to 128.
00012     ht->hash_size     = 0;   // Initially, the hash table is empty.
00013     CREATE_MEMORY(ht->hash_table, ht->hash_capacity);
00014
00015     // Setting all slots to -1 (indicating empty slots).
00016     for (int i = 0; i < ht->hash_capacity; i++) {
00017         ht->hash_table[i] = -1;
00018     }
00019
00020     // Setting the element size.
00021     ht->element_size = element_size;
00022     return ht;
00023 }
00024
00025 void delete_hash_table(hash_table *ht) {
00026     if (ht == NULL) {
00027         return;
00028     }
00029     free(ht->hash_table);
00030     free(ht);
00031 }
00032
00033 // Hash function using a simple polynomial rolling hash. Hashes an array of
00034 // integers.
00035 static int
00036 hash_function(int *elements,
00037               int size,
00038               int hash_capacity
00039 ) {
00040     int hash = 0;
00041     int a    = 0x9e3779b9; // fractional bits of the golden ratio
```

```
00042     for (int j = 0; j < size; j++) {
00043         hash = (hash * (size + 1) + (elements[j]) * a) % hash_capacity;
00044     }
00045     return hash;
00046 }
00047
00048 // Rehashing function to resize the hash table when load factor exceeds 0.5
00049 static void hash_table_rehash(hash_table *ht, int *elements) {
00050     // Saving the pointer to the old table and its size.
00051     int *old_table = ht->hash_table;
00052     int old_size   = ht->hash_capacity;
00053
00054     // Doubling the size of the hash table.
00055     ht->hash_capacity *= 2;
00056     CREATE_MEMORY(ht->hash_table, ht->hash_capacity);
00057     for (int i = 0; i < ht->hash_capacity; i++) {
00058         ht->hash_table[i] = -1;
00059     }
00060
00061     // Rehashing the elements from the old table to the new one.
00062     for (int i = 0; i < old_size; i++) {
00063         if (old_table[i] == -1) {
00064             continue; // Skip empty slots
00065         }
00066         // Computing the index correspinding to the element with the hash
00067         // function.
00068         int index = hash_function(elements + old_table[i] * (ht->element_size),
00069                                   ht->element_size, ht->hash_capacity);
00070
00071         // linear probing to find the next available slot
00072         int j     = 0;
00073         while (true) {
00074             int new_index = (index + j) % ht->hash_capacity; // Linear probing
00075             if (ht->hash_table[new_index] == -1) {
00076                 ht->hash_table[new_index] = old_table[i];
00077                 break; // Successfully inserted
00078             } else {
00079                 j++;
00080             }
00081         }
00082     }
00083     free(old_table);
00084 }
00085
00086 // Compares two array of integers to check if they are equal
00087 static bool hash_array_compare(int *a, int *b, int size) {
00088     for (int i = 0; i < size; i++) {
00089         if (a[i] != b[i]) {
00090             return false;
00091         }
00092     }
00093     return true;
00094 }
00095
00096 bool hash_table_insert(hash_table *ht, int *elements, int i, int *ret) {
00097     // Rehashing if the load factor exceeds 0.5
00098     if (ht->hash_size >= (ht->hash_capacity » 1)) {
00099         hash_table_rehash(ht, elements);
```

```
00100     }
00101
00102     // Computing the index where the element should be.
00103     int index = hash_function(elements + i * (ht->element_size),
00104                               ht->element_size, ht->hash_capacity);
00105
00106     // Linear probing to find either the element or a free spot
00107     int s     = 0;
00108     while (true) {
00109         int s_index = (index + s) % ht->hash_capacity; // Linear probing
00110         if (ht->hash_table[s_index] == -1) {
00111             // If we find an empty spot, we insert the value and return true to
00112             // indicate that the insertion was performed.
00113             ht->hash_table[s_index] = i;
00114             ht->hash_size++;
00115             return true;
00116         } else if (hash_array_compare(elements + i * (ht->element_size),
00117                                       elements + ht->hash_table[s_index] *
00118                                                  (ht->element_size),
00119                                       ht->element_size)) {
00120             if (ret) {
00121                 *ret = ht->hash_table[s_index];
00122             }
00123             // If the value is already present, we do not insert it and return
00124             // false to indicate that the insertion failed.
00125             return false;
00126         } else {
00127             // If the slot is occupied by a different value, we try the next
00128             // slot.
00129             s++;
00130         }
00131     }
00132     return false;
00133 }
```

4.53 Référence du fichier type_hash.h

Implémentation des tables de hachage (déjà écrite, à utiliser pour les parcours de graphes).

```
#include "type_basic.h"
#include <stdbool.h>
```

Structures de données

— struct hash_table

Type utilisé pour représenter une table de hachage.

Fonctions

— hash_table * create_hash_table (int element_size)

Crée une nouvelle table de hachage vide.

— void delete_hash_table (hash_table *ht)

Supprime une table de hachage.

— bool hash_table_insert (hash_table *ht, int *elements, int i, int *ret)

Tente d'insérer un élément dans la table de hachage.

4.53.1 Description détaillée

Implémentation des tables de hachage (déjà écrite, à utiliser pour les parcours de graphes).

Ce fichier contient une implémentation déjà écrite des tables de hachage. Vous ne devez PAS LE MODIFIER. vous pouvez l'utiliser dans votre code pour stocker un ensemble détats déjà visités quand vous faites un parcours de graphe.

Remarques

La table de hachage est similaire à celle fournie dans le projet d'algorithmique. Elle ne peut stocker que des éléments encodés comme des tableaux d'entiers. Cela signifie que pour stocker des objets, vous devrez les encoder sous forme de tableaux d'entiers. De plus, la table de hachage ne stocke pas les éléments eux-mêmes, mais des indices vers un tableau de stockage externe que vous devez gérer vous-même.

Définition dans le fichier type_hash.h.

4.53.2 Documentation des fonctions

4.53.2.1 create_hash_table()

```
hash_table * create_hash_table (
          int element_size)
```

Crée une nouvelle table de hachage vide.

Renvoie

La table de hachage vide créée.

Paramètres

element_size	Nombre de valeurs utilisées pour encoder un seul élément dans le tableau de stockage externe.

Définition à la ligne 5 du fichier type_hash.c.

4.53.2.2 delete_hash_table()

```
void delete_hash_table (
            hash_table * ht)
```

Supprime une table de hachage.

Paramètres

ht	La table de hachage à supprimer.

Définition à la ligne 25 du fichier type_hash.c.

4.53.2.3 hash_table_insert()

```
bool hash_table_insert (
            hash_table * ht,
            int * elements,
            int i,
            int * ret)
```

Tente d'insérer un élément dans la table de hachage.

Si l'élément est déjà présent dans la table de hachage, l'insertion n'est pas effectuée. De plus l'indice du premier élément identique trouvé est stocké dans la variable pointée par ret (si ret n'est pas NULL).

Remarques

L'indice i fait référence à la position de l'élément dans le tableau de stockage externe. Puisque chaque élément est encodé comme une séquence d'entiers de taille fixe (le champ element↩ _size), la table de hachage va en fait examiner la séquence de element_size valeurs entières commençant à la position i * element_size dans le tableau de stockage externe.

Renvoie

true si l'élément a été inséré, false s'il était déjà présent. Dans ce dernier cas, la table de hachage n'est pas modifiée.

Paramètres

ht	La table de hachage.
elements	Tableau de stockage externe contenant tous les éléments.
i	Indice de l'élément à insérer dans le tableau de stockage externe.
ret	Pointeur vers un entier où sera stocké l'indice de l'élément existant (si déjà présent).

Définition à la ligne 96 du fichier type_hash.c.

4.54 type_hash.h

Aller à la documentation de ce fichier.

```
00001
00017
00018 #ifndef TYPE_HASH_H
00019 #define TYPE_HASH_H
00020
00021 #include "type_basic.h"
00022 #include <stdbool.h>
00023
00039 typedef struct
00040 {
00041     int hash_capacity;
00042     int hash_size;
00043     int *hash_table;
00044     int element_size;
00045 } hash_table;
00046
00054 hash_table *create_hash_table(int element_size
00055 );
00056
00061 void delete_hash_table(hash_table *ht
00062 );
00063
00082 bool hash_table_insert(hash_table *ht,
00083                        int *elements,
00084                        int i,
00085                        int *ret
00086 );
00087
00088 #endif
```

Index

ADVANCED
    parameters.h, 77
alloc.c, 19
    check_null, 20
    multiple_free, 20
alloc.h, 21
    CHECK_NULL, 25
    check_null, 26
    CREATE_MEMORY, 23
    CREATE_ZERO_MEMORY, 23
    multiple_free, 26
    REDIMENSION_MEMORY, 24
and_barray
    type_boolarray.c, 102
    type_boolarray.h, 106
array
    barray, 6
ATTENTION
    error.h, 31

barray, 5
    array, 6
    size, 6
    size_array, 6

CHAR
    regexp.h, 88
CHECK_NULL
    alloc.h, 25
check_null
    alloc.c, 20
    alloc.h, 26
COMPLEMENT
    regexp.h, 88
CONCAT
    regexp.h, 89
copy_barray
    type_boolarray.c, 102
    type_boolarray.h, 107
create_barray
    type_boolarray.c, 100
    type_boolarray.h, 105
create_emptylang
    nfa.c, 50
    nfa.h, 58
create_hash_table
    type_hash.c, 109
    type_hash.h, 115
CREATE_MEMORY
    alloc.h, 23
create_sing_epsilon
    nfa.c, 51
    nfa.h, 58
create_sing_letter
    nfa.c, 51
    nfa.h, 59
CREATE_ZERO_MEMORY
    alloc.h, 23
CRITICAL
    error.h, 36, 37
CRITICAL_L
    error.h, 37

DEBUG
    error.h, 35, 37
DEBUG_L
    error.h, 37
DEBUG_LEVEL
    error.h, 31
debug_level
    error.h, 36
delete_barray
    type_boolarray.c, 100
    type_boolarray.h, 105
delete_hash_table
    type_hash.c, 109
    type_hash.h, 116

edge_list_to_ngraph
    graphs.c, 40
    graphs.h, 44
element_size
    hash_table, 8
EMPTY
    regexp.h, 88
eps_trans
    nfa, 11
EPSILON

regexp.h, 88
ERROR
error.h, 36, 37
error.h, 28
ATTENTION, 31
CRITICAL, 36, 37
CRITICAL_L, 37
DEBUG, 35, 37
DEBUG_L, 37
DEBUG_LEVEL, 31
debug_level, 36
ERROR, 36, 37
ERROR_L, 37
FATAL, 34
HINT, 31
INFO, 35, 37
INFO_L, 37
KO, 31
msg_level, 37
OFF, 37
OK, 31
PRINT_DEBUG, 34
PRINT_ERROR, 32
TRACE, 34, 37
TRACE_L, 37
WARNING, 35, 37
WARNING_L, 37
ERROR_L
error.h, 37

FATAL
error.h, 34
finals
nfa, 12
from
one_edge, 15

getsize_barray
type_boolarray.c, 100
type_boolarray.h, 105
getval_barray
type_boolarray.c, 101
type_boolarray.h, 106
graphs.c, 39
edge_list_to_ngraph, 40
ngraph_to_edge_list, 39
ngraph_to_multi_edges, 41
graphs.h, 42
edge_list_to_ngraph, 44
ngraph, 43
ngraph_to_edge_list, 44
ngraph_to_multi_edges, 45
grep
grep.c, 47
grep.h, 48
grep.c, 47
grep, 47
grep.h, 48
grep, 48

hash_array_compare
type_hash.c, 111
hash_capacity
hash_table, 7
hash_function
type_hash.c, 109
hash_size
hash_table, 7
hash_table, 6
element_size, 8
hash_capacity, 7
hash_size, 7
hash_table, 8
hash_table_insert
type_hash.c, 111
type_hash.h, 116
hash_table_rehash
type_hash.c, 111
HINT
error.h, 31

in
multi_edge, 9
INFO
error.h, 35, 37
INFO_L
error.h, 37
initials
nfa, 12
INTER
regexp.h, 88
intervals
ngraph, 14

KO
error.h, 31

lab
multi_edge, 9
label
one_edge, 16
left
regexp, 17
letter
regexp, 17

letter_names
nfa, 12

max
type_basic.h, 97
min
type_basic.h, 97
msg_level
error.h, 37
multi_edge, 8
in, 9
lab, 9
out, 9
multiple_free
alloc.c, 20
alloc.h, 26

nb_finals
nfa, 12
nb_initials
nfa, 12
nb_labels
ngraph, 14
nb_letters
nfa, 11
nb_states
nfa, 11
ngraph, 14
nfa, 10
eps_trans, 11
finals, 12
initials, 12
letter_names, 12
nb_finals, 12
nb_initials, 12
nb_letters, 11
nb_states, 11
state_names, 13
transitions, 11
nfa.c, 49
create_emptylang, 50
create_sing_epsilon, 51
create_sing_letter, 51
nfa_compute_runs, 54
nfa_concat, 52
nfa_copy, 52
nfa_delete, 51
nfa_elimeps, 54
nfa_mirror, 53
nfa_reset_state_names, 50
nfa_star, 53
nfa_trim, 53
nfa_union, 52
nfa.h, 56
create_emptylang, 58
create_sing_epsilon, 58
create_sing_letter, 59
nfa_compute_runs, 62
nfa_concat, 60
nfa_copy, 59
nfa_delete, 59
nfa_elimeps, 61
nfa_mirror, 61
nfa_reset_state_names, 58
nfa_star, 60
nfa_trim, 61
nfa_union, 60
nfa_brzozowski
nfa_minimization.c, 74
nfa_minimization.h, 76
nfa_complement
nfa_determi.c, 65
nfa_determi.h, 67
nfa_compute_runs
nfa.c, 54
nfa.h, 62
nfa_concat
nfa.c, 52
nfa.h, 60
nfa_copy
nfa.c, 52
nfa.h, 59
nfa_delete
nfa.c, 51
nfa.h, 59
nfa_determi.c, 64
nfa_complement, 65
nfa_determinize, 65
nfa_determi.h, 66
nfa_complement, 67
nfa_determinize, 67
nfa_determinize
nfa_determi.c, 65
nfa_determi.h, 67
nfa_elimeps
nfa.c, 54
nfa.h, 61
nfa_intersec.c, 68
nfa_intersect, 69
nfa_intersec.h, 70
nfa_intersect, 71
nfa_intersect
nfa_intersec.c, 69

nfa_intersec.h, 71
nfa_mccluskey
nfa_mccluskey.c, 72
nfa_mccluskey.h, 73
nfa_mccluskey.c, 72
nfa_mccluskey, 72
nfa_mccluskey.h, 73
nfa_mccluskey, 73
nfa_minimization.c, 74
nfa_brzozowski, 74
nfa_minimization.h, 75
nfa_brzozowski, 76
nfa_mirror
nfa.c, 53
nfa.h, 61
nfa_reset_state_names
nfa.c, 50
nfa.h, 58
nfa_star
nfa.c, 53
nfa.h, 60
nfa_trim
nfa.c, 53
nfa.h, 61
nfa_union
nfa.c, 52
nfa.h, 60
ngraph, 13
graphs.h, 43
intervals, 14
nb_labels, 14
nb_states, 14
to_array, 14
ngraph_to_edge_list
graphs.c, 39
graphs.h, 44
ngraph_to_multi_edges
graphs.c, 41
graphs.h, 45
NONE
regexp.h, 89

OFF
error.h, 37
OK
error.h, 31
one_edge, 15
from, 15
label, 16
to, 15
op
regexp, 17
or_barray
type_boolarray.c, 101
type_boolarray.h, 106
out
multi_edge, 9

parameters.h, 76
ADVANCED, 77
PRINT_DEBUG
error.h, 34
PRINT_ERROR
error.h, 32

REDIMENSION_MEMORY
alloc.h, 24
reg_complement
regexp.c, 81
regexp.h, 91
reg_concat
regexp.c, 81
regexp.h, 91
reg_copy
regexp.c, 79
regexp.h, 89
reg_empty
regexp.c, 79
regexp.h, 89
reg_epsilon
regexp.c, 79
regexp.h, 90
reg_free
regexp.c, 79
regexp.h, 89
reg_inter
regexp.c, 80
regexp.h, 90
reg_issimple
regexp.c, 78
regexp.h, 89
reg_letter
regexp.c, 80
regexp.h, 90
reg_print
regexp.c, 82
regexp.h, 92
reg_print_helper
regexp.c, 82
reg_star
regexp.c, 81
regexp.h, 91
reg_thompson
regexp_thompson.c, 94

regexp_thompson.h, 95
reg_union
regexp.c, 80
regexp.h, 90
regelem
regexp.h, 88
regexp, 16
left, 17
letter, 17
op, 17
regexp.h, 88
right, 17
regexp.c, 77
reg_complement, 81
reg_concat, 81
reg_copy, 79
reg_empty, 79
reg_epsilon, 79
reg_free, 79
reg_inter, 80
reg_issimple, 78
reg_letter, 80
reg_print, 82
reg_print_helper, 82
reg_star, 81
reg_union, 80
regexp.h, 86
CHAR, 88
COMPLEMENT, 88
CONCAT, 89
EMPTY, 88
EPSILON, 88
INTER, 88
NONE, 89
reg_complement, 91
reg_concat, 91
reg_copy, 89
reg_empty, 89
reg_epsilon, 90
reg_free, 89
reg_inter, 90
reg_issimple, 89
reg_letter, 90
reg_print, 92
reg_star, 91
reg_union, 90
regelem, 88
regexp, 88
STAR, 89
UNION, 88
regexp_thompson.c, 94
reg_thompson, 94
regexp_thompson.h, 95
reg_thompson, 95
right
regexp, 17

setfalse_barray
type_boolarray.c, 101
type_boolarray.h, 106
settrue_barray
type_boolarray.c, 101
type_boolarray.h, 105
size
barray, 6
size_array
barray, 6
STAR
regexp.h, 89
state_names
nfa, 13

to
one_edge, 15
to_array
ngraph, 14
TRACE
error.h, 34, 37
TRACE_L
error.h, 37
transitions
nfa, 11
type_basic.h, 96
max, 97
min, 97
uchar, 98
uint, 98
ulong, 98
ushort, 98
type_boolarray.c, 99
and_barray, 102
copy_barray, 102
create_barray, 100
delete_barray, 100
getsize_barray, 100
getval_barray, 101
or_barray, 101
setfalse_barray, 101
settrue_barray, 101
type_boolarray.h, 103
and_barray, 106
copy_barray, 107
create_barray, 105

delete_barray, 105
getsize_barray, 105
getval_barray, 106
or_barray, 106
setfalse_barray, 106
settrue_barray, 105
type_hash.c, 108
create_hash_table, 109
delete_hash_table, 109
hash_array_compare, 111
hash_function, 109
hash_table_insert, 111
hash_table_rehash, 111
type_hash.h, 114
create_hash_table, 115
delete_hash_table, 116
hash_table_insert, 116

uchar
type_basic.h, 98
uint
type_basic.h, 98
ulong
type_basic.h, 98
UNION
regexp.h, 88
ushort
type_basic.h, 98

WARNING
error.h, 35, 37
WARNING_L
error.h, 37